

Master 2 Informatique
Data sciences, infrastructure cloud et sécurité

Machine Learning By Practice

Fabrice LAURI
Maître de conférences
à l'Université de Technologie de Belfort-Montbéliard

General Overview

- Introduction
- Data Visualization
- Dimensionality Reduction
- **Supervised Learning**
- Unsupervised Learning
- Reinforcement Learning
- Conclusion

CM #2

Supervised Learning

- Preliminaries
- Machine Learning API and basics
- Linear discriminants
- Neurons, Perceptron and Multi-Layer Perceptron (MLP)
- Deep Neural Networks (DNN)

Summary of the categories of *Machine Learning*

Supervised Learning

Models that can predict labels based on labeled training data

Classification

Models that predict labels as two or more discrete categories

Regression

Models that predict continuous labels

Unsupervised Learning

Models that identify structure in unlabeled data

Clustering

Models that detect and identify distinct groups in the data

Dimensionality Reduction

Models that detect and identify lower-dimensional structure in higher-dimensional data

Supervised Learning

Supervised Learning

Preliminaries

Definitions

- **Inputs:** an input vector is the data given as one input to the system. A vector $\mathbf{x}=(x_1, x_2, \dots, x_m)$ has m **features** x_i .
- **Targets:** the target vector $\mathbf{t}=(t_1, t_2, \dots, t_n)$ are the extra data we need for supervised learning. They provide the correct answers that the algorithm is learning about.
- **Data set:** a data set is denoted by $\{(x^{(i)}, t^{(i)})\}_{i=1, \dots, N}$, where $x^{(i)}$ is the i -th input and $t^{(i)}$ is the associated target.
- **Outputs:** The output vector is $\mathbf{y}=(y_1, y_2, \dots, y_n)$ with n elements y_i .
- **Weights:** w_{ij} is the weighted connection between nodes i and j in a (neural) network. These weights are equivalent to the synapses in the brain. They are arranged into a matrix $\mathbf{W}$.
- **Error:** a function that computes the inaccuracies of the learning system. It is a function of the outputs $\mathbf{y}$ and targets $\mathbf{t}$.

Supervised Learning Preliminaries

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

Here each row of the data refers to a single observed flower, and the number of rows is the total number of flowers in the dataset.

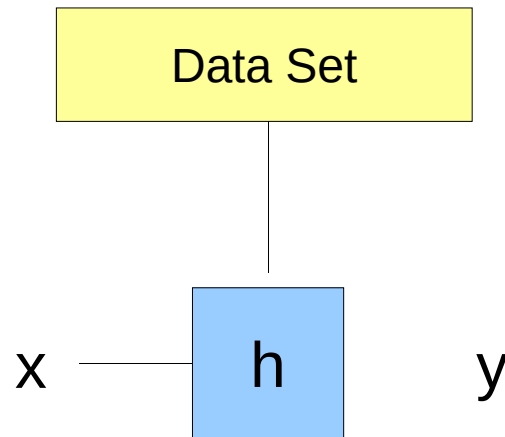
In general, we will refer to the rows of the matrix as samples, and the number of rows as n_{samples} .

Each column of the data refers to a particular quantitative piece of information that describes each sample. The columns of the matrix are the features, and the number of columns are referred to as n_{features} .

Feature
Matrix

Target
Vector

Supervised Learning Preliminaries

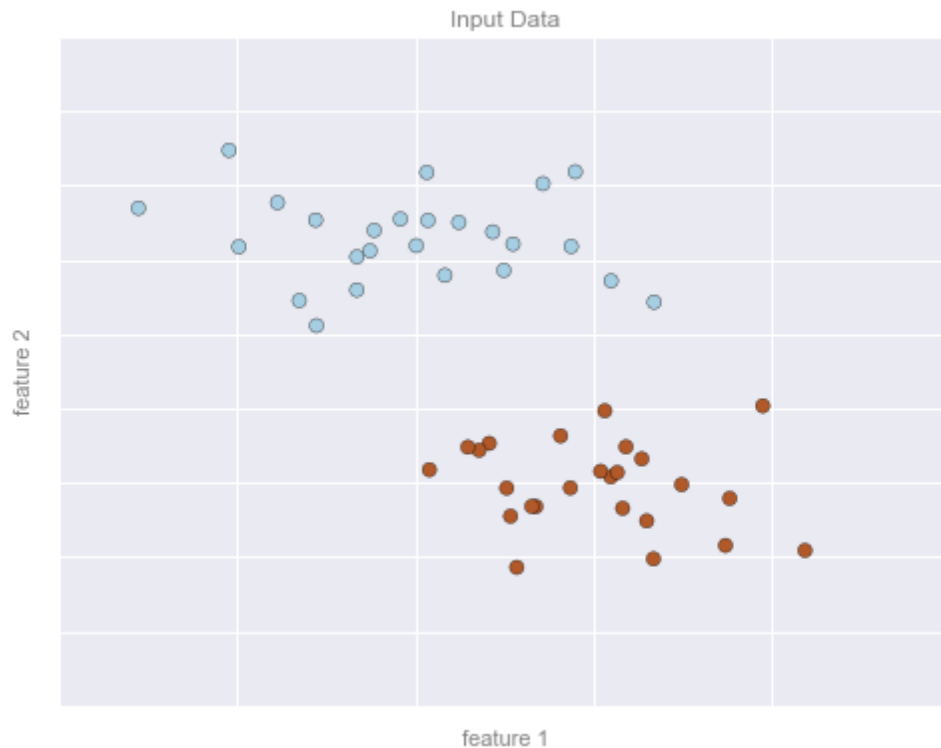


Regression: $y \in \mathbb{R}$
Classification: $y \in \mathbb{N}$

Machine Learning in general, and Supervised Learning in particular, should be able to **generalise**.

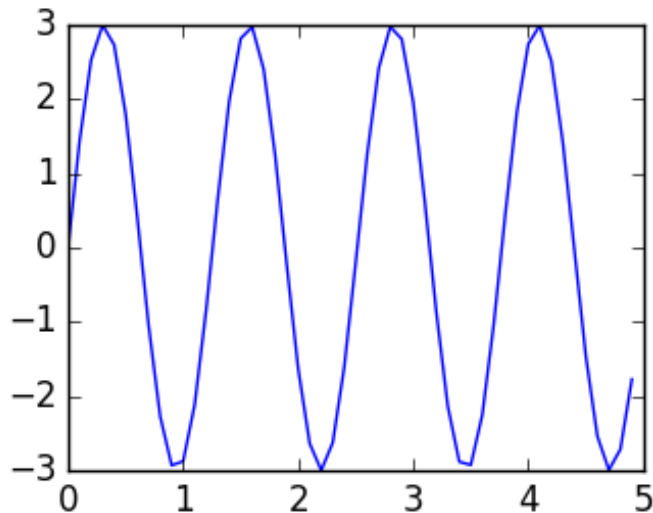
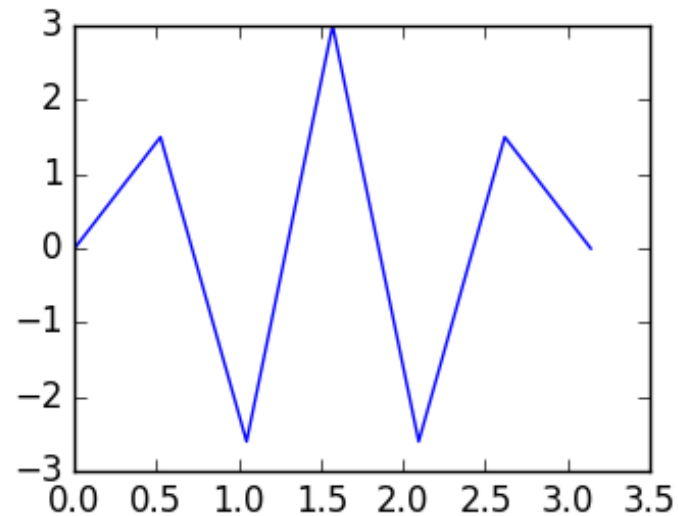
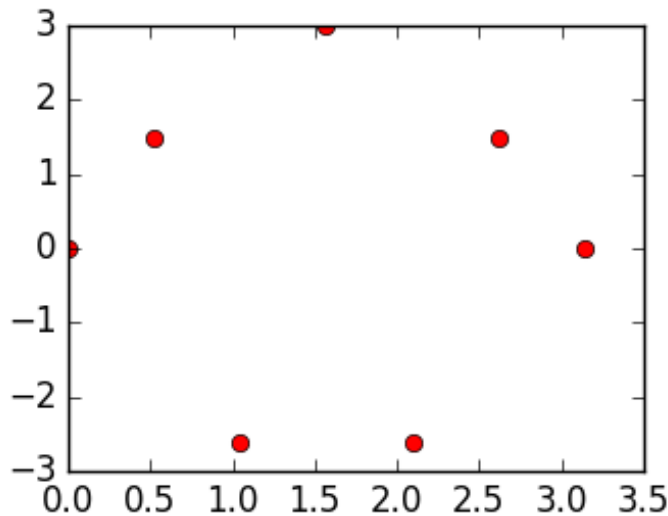
Supervised Learning Classification

Classification = Find a line/curve/boundary that separates out the data.



Supervised Learning Regression

Regression = Fit a line/curve to the data.



Model validation

Model Validation

Model validation is very simple: after choosing a model and its hyperparameters, we can estimate how effective it is by applying it to some of the training data and comparing the prediction to the known value.

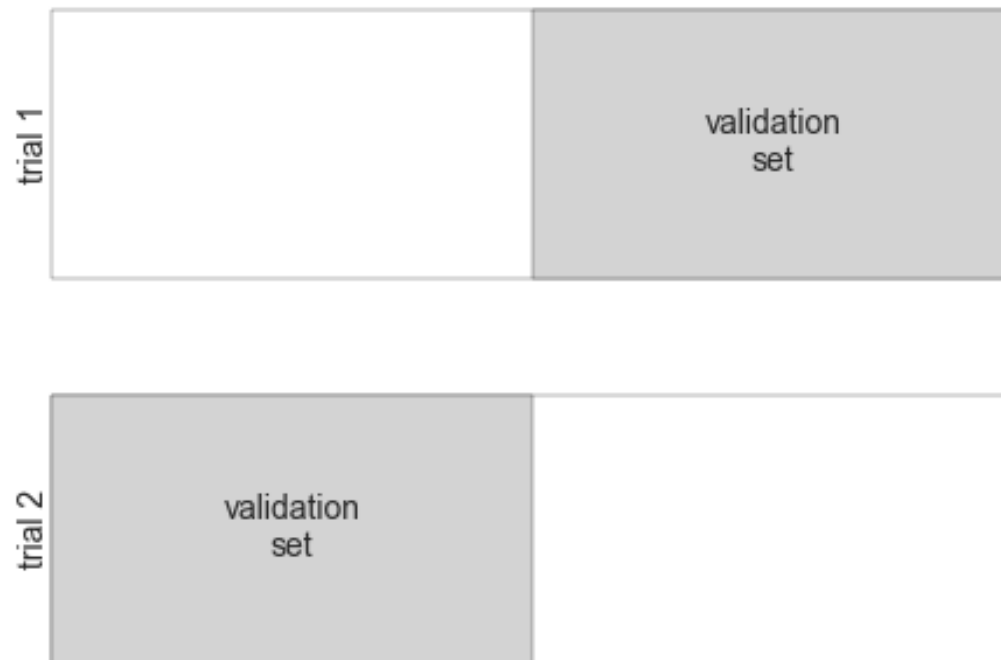
This is the wrong way...

The right way is to use holdout sets: some subset of the data are hold back from the training of the model, and then use this holdout set to check the model performance.

Model Validation: cross validation

The problem of using a holdout set for model validation is that we have lost a portion of our data to the model training. In the previous case, half the dataset does not contribute to the training of the model! This can cause problems, especially if the initial set of training data is small.

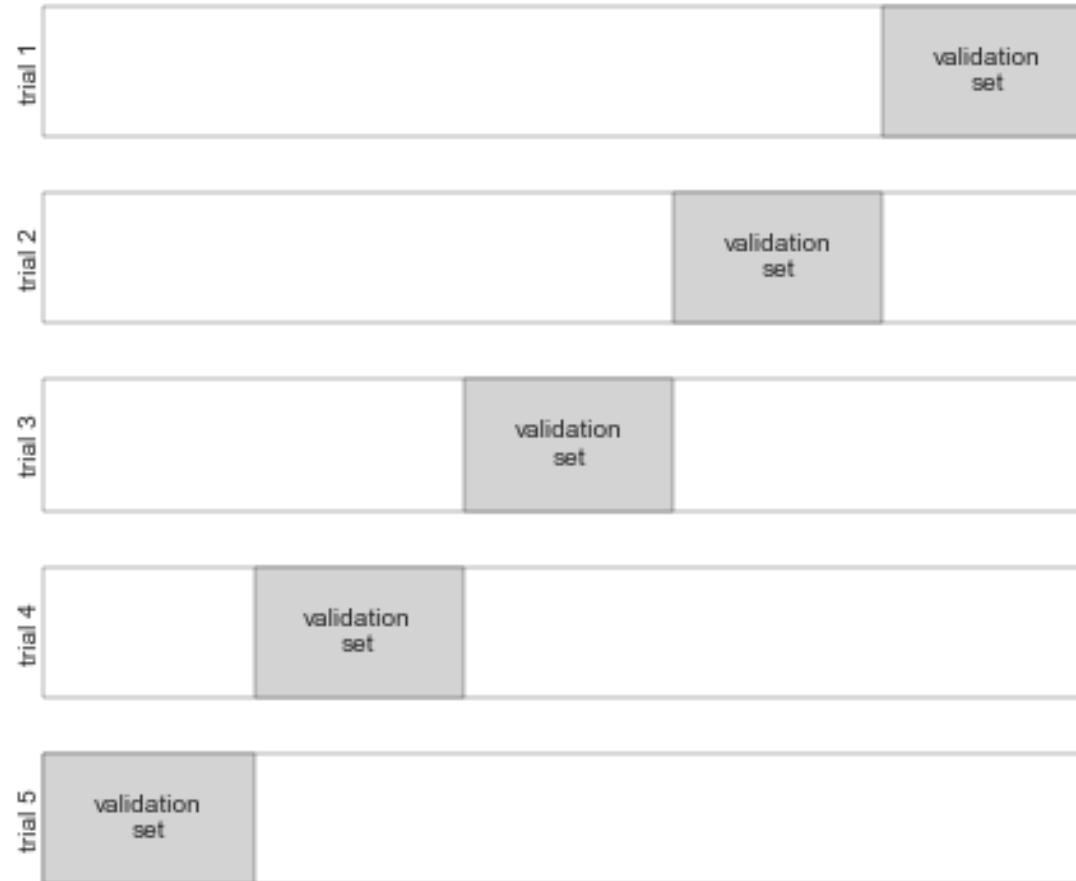
One way to address this is to use **cross-validation**; that is, to do a sequence of fits where each subset of the data is used both as a training set and as a validation set. Visually, it might look something like this:



Two-fold cross-validation

Model Validation: cross validation

We could expand on this idea to use even more trials, and more folds in the data—for example, here is five-fold cross-validation:



Model Validation:

Leave-one-out cross validation & co.

We might wish to go to the extreme case in which our number of folds is equal to the number of data points: that is, we train on all points but one in each trial. This type of cross-validation is known as leave-one-out cross validation.

Several other cross validation methods can be used. See the documentation of Scikit-Learn...

Selecting the best model

Of core importance is the following question: if our estimator is underperforming, how should we move forward? There are several possible answers:

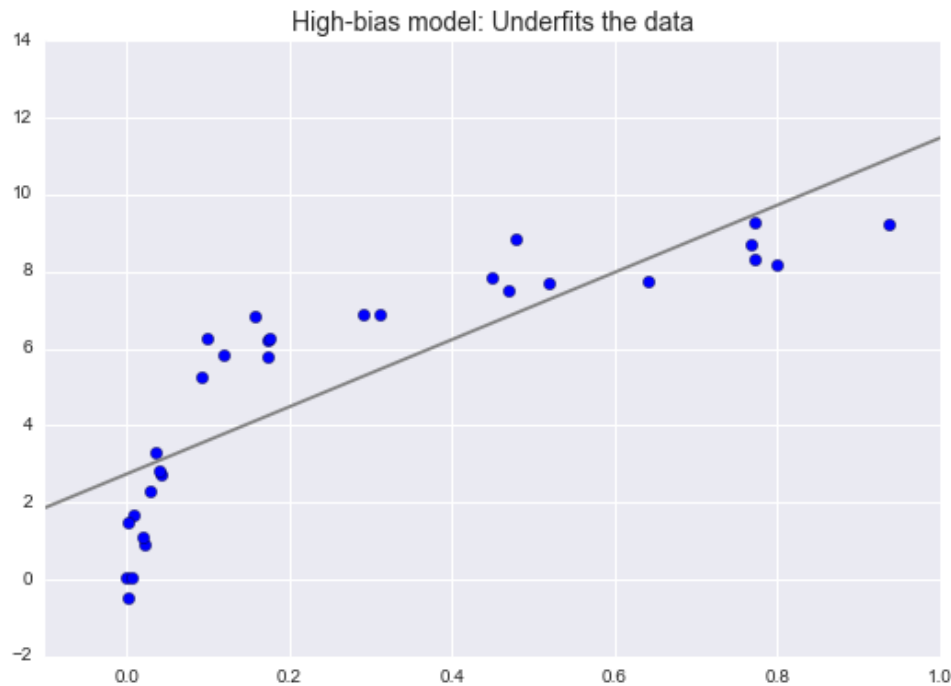
- Use a more complicated/more flexible model
- Use a less complicated/less flexible model
- Gather more training samples
- Gather more data to add features to each sample

The answer to this question is often counter-intuitive.

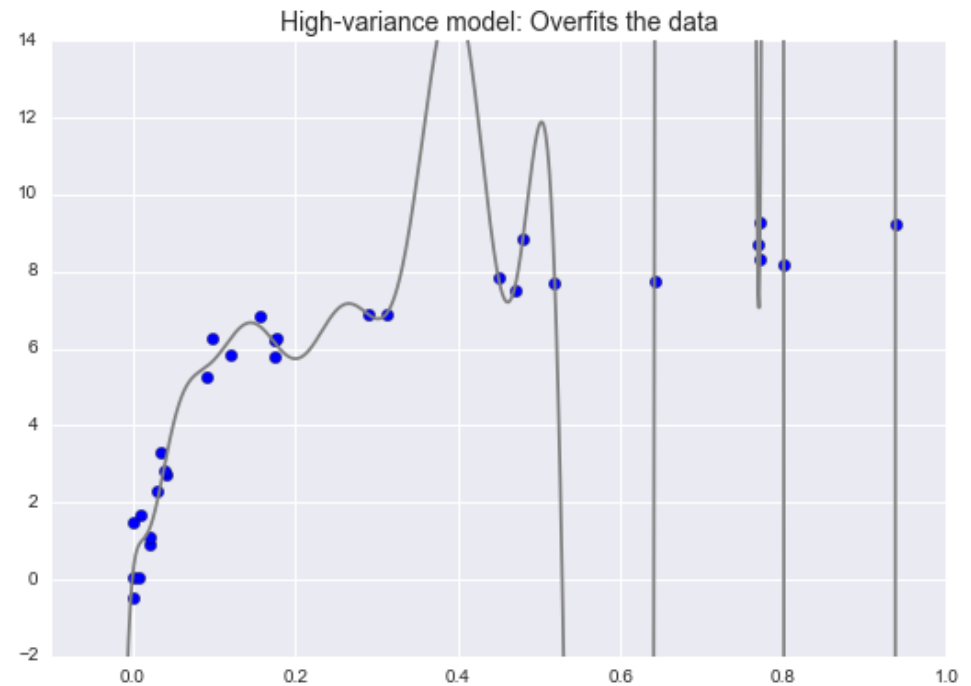
In particular, sometimes using a more complicated model will give worse results, and adding more training samples may not improve your results!

Tradeoff bias/variance

The question of "the best model" consists in finding a sweet spot in the tradeoff between bias and variance. Consider the following figure, which presents two regression fits to the same dataset:



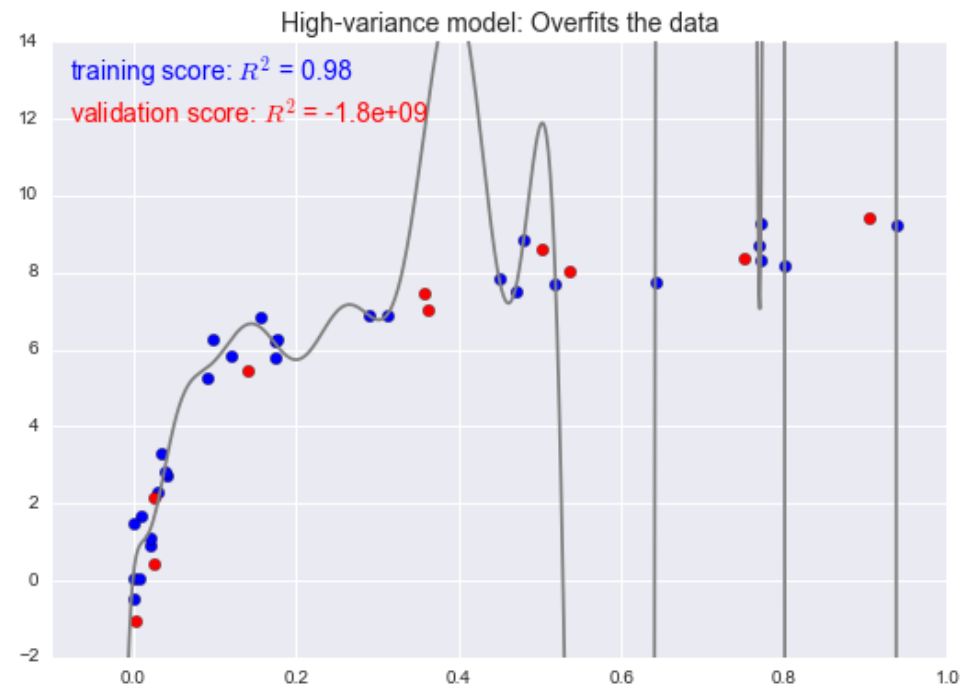
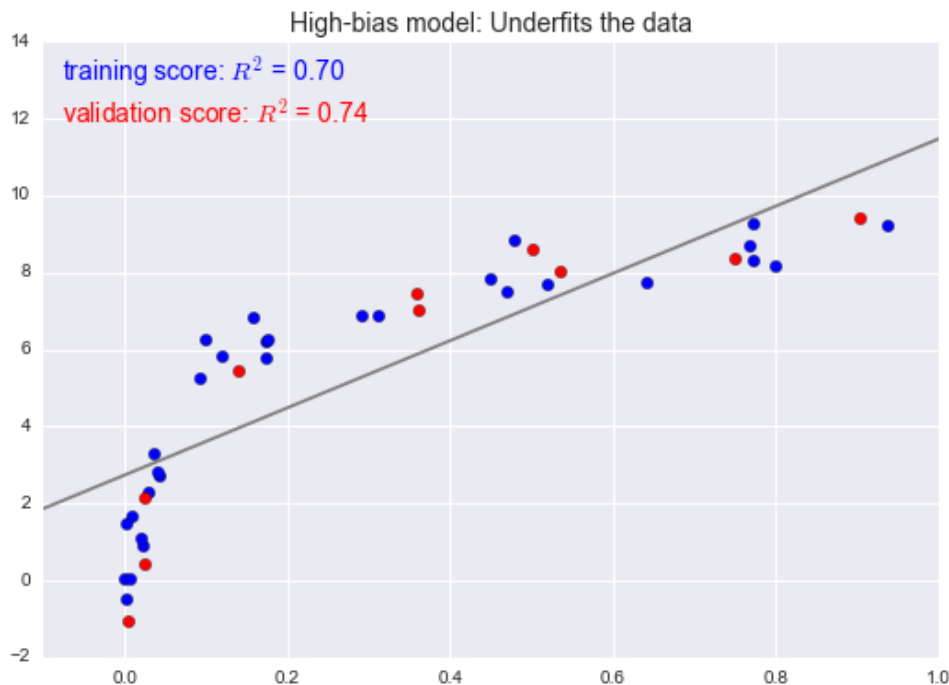
Such a model is said to underfit the data: that is, it does not have enough model flexibility to suitably account for all the features in the data; another way of saying this is that the model has high bias.



Such a model is said to overfit the data: that is, it has so much model flexibility that the model ends up accounting for random errors as well as the underlying data distribution; another way of saying this is that the model has high variance.

Tradeoff bias/variance

To look at this in another light, consider what happens if we use these two models to predict the y-value for some new data. In the following diagrams, the red/lighter points indicate data that is omitted from the training set:

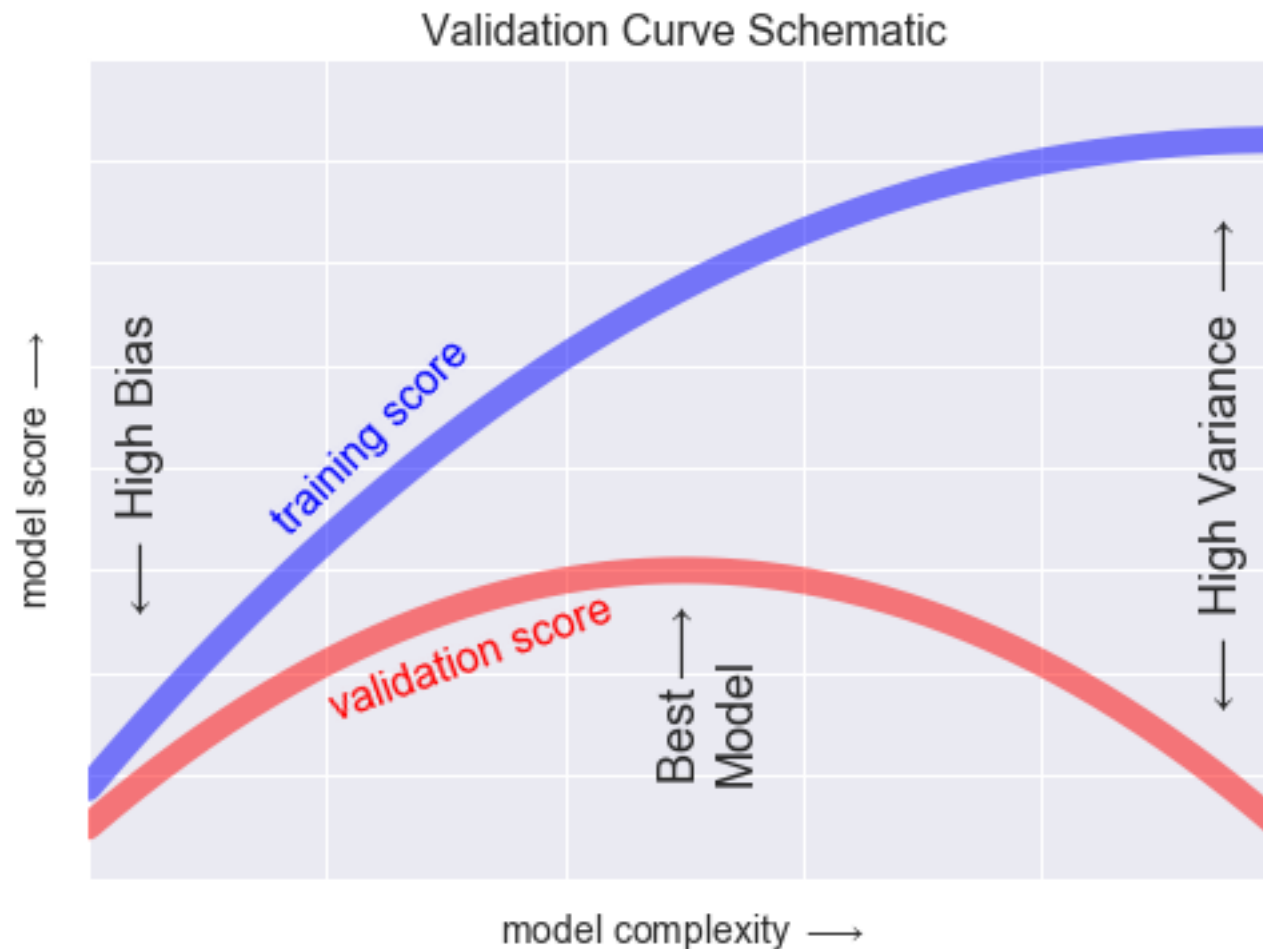


The score here is the R^2 score (coefficient of determination). It measures how well a model performs relative to a simple mean of the target values.

$R^2=1$ indicates a perfect match, $R^2=0$ indicates the model does no better than simply taking the mean of the data, and negative values mean even worse models.

Validation Curve

If we imagine that we have some ability to tune the model complexity, we would expect the training score and validation score to behave as illustrated in the following figure:



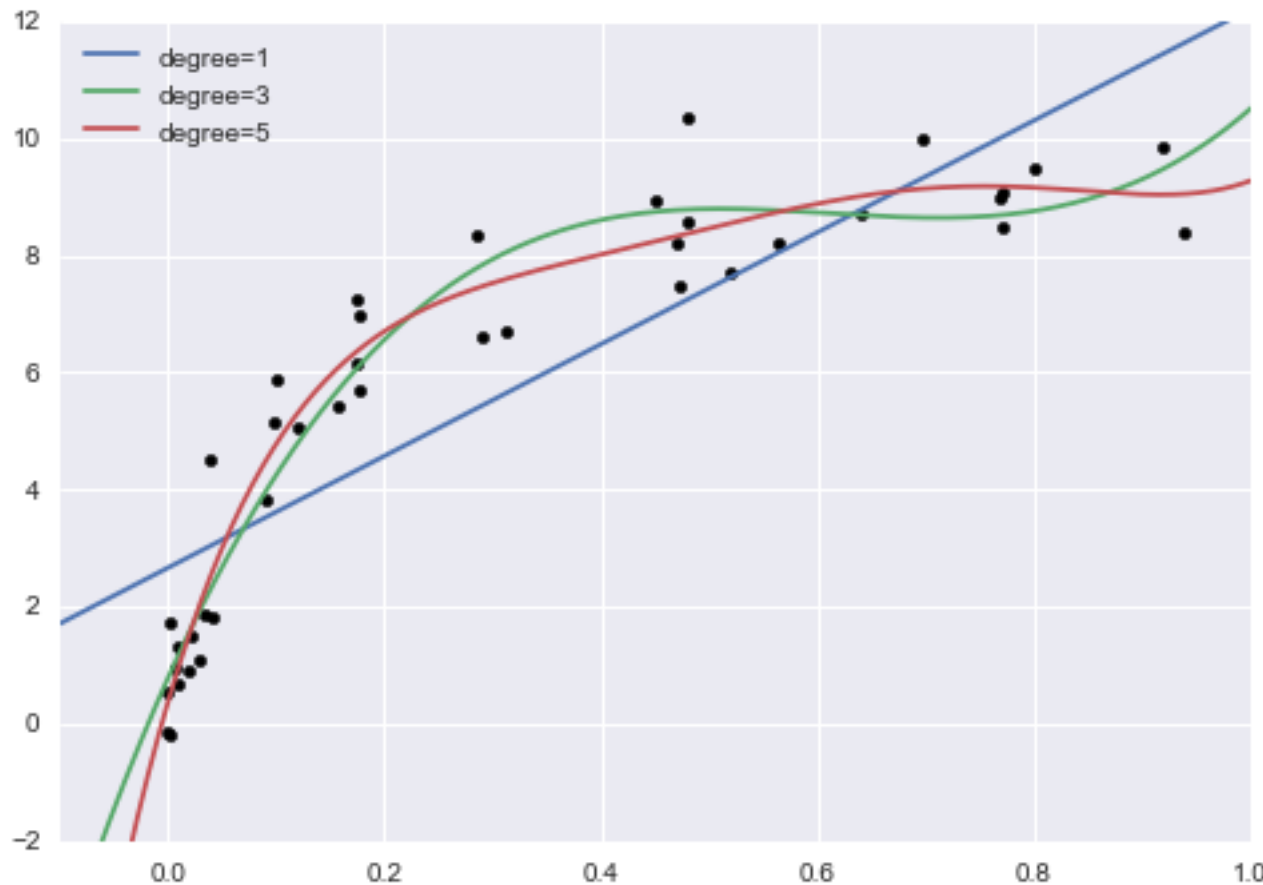
Validation Curve

From the diagram shown before we see the following essential features:

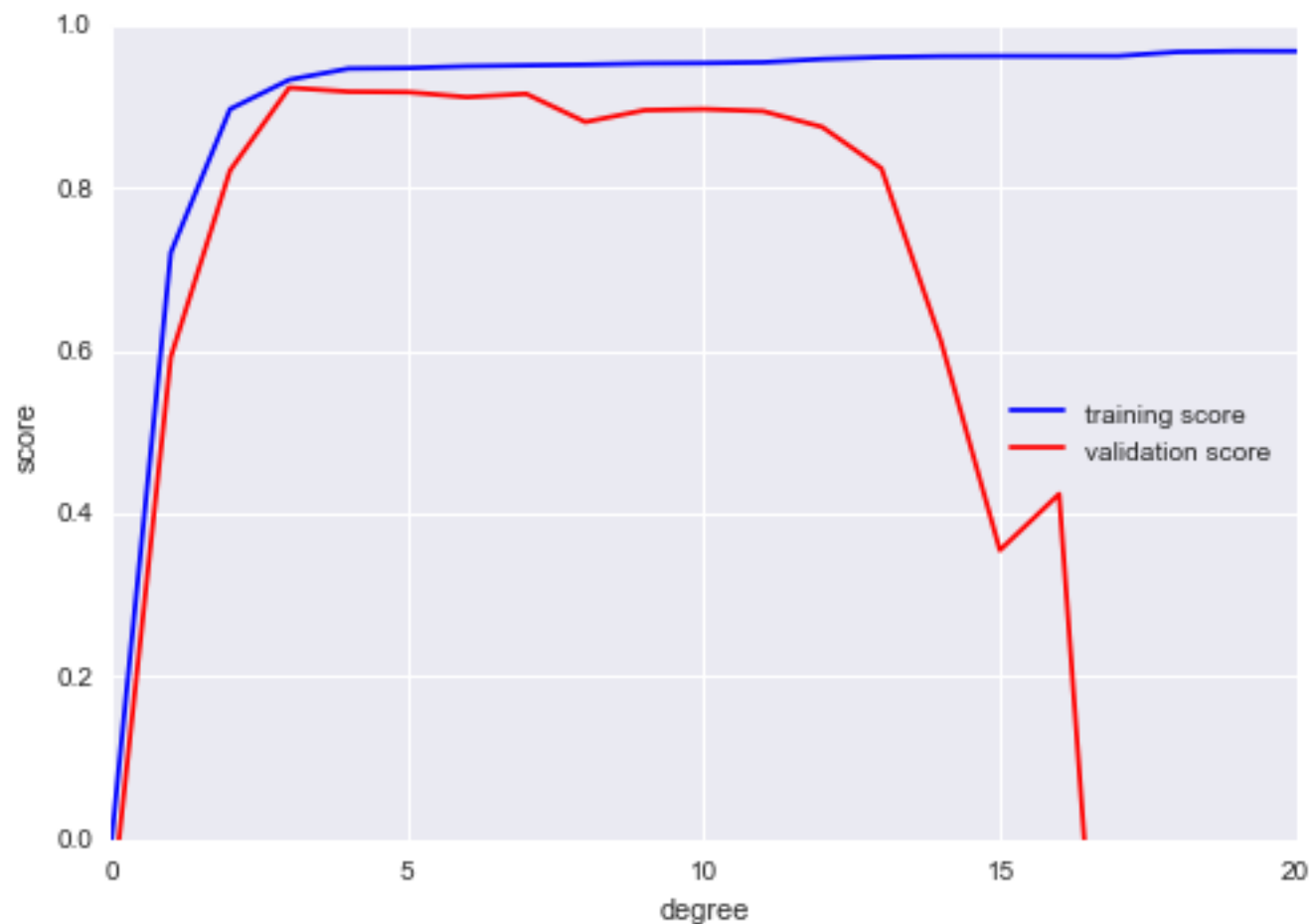
- The training score is everywhere higher than the validation score. This is generally the case: the model will be a better fit to data it has seen than to data it has not seen.
- For very low model complexity (a high-bias model), the training data is under-fit, which means that the model is a poor predictor both for the training data and for any previously unseen data.
- For very high model complexity (a high-variance model), the training data is over-fit, which means that the model predicts the training data very well, but fails for any previously unseen data.
- For some intermediate value, the validation curve has a maximum. This level of complexity indicates a suitable trade-off between bias and variance.

Example: polynomial regression model

The knob controlling model complexity in this case is the degree of the polynomial, which can be any non-negative integer. A useful question to answer is this: what degree of polynomial provides a suitable trade-off between bias (under-fitting) and variance (over-fitting)?



Example: validation curve for the polynomial regression model

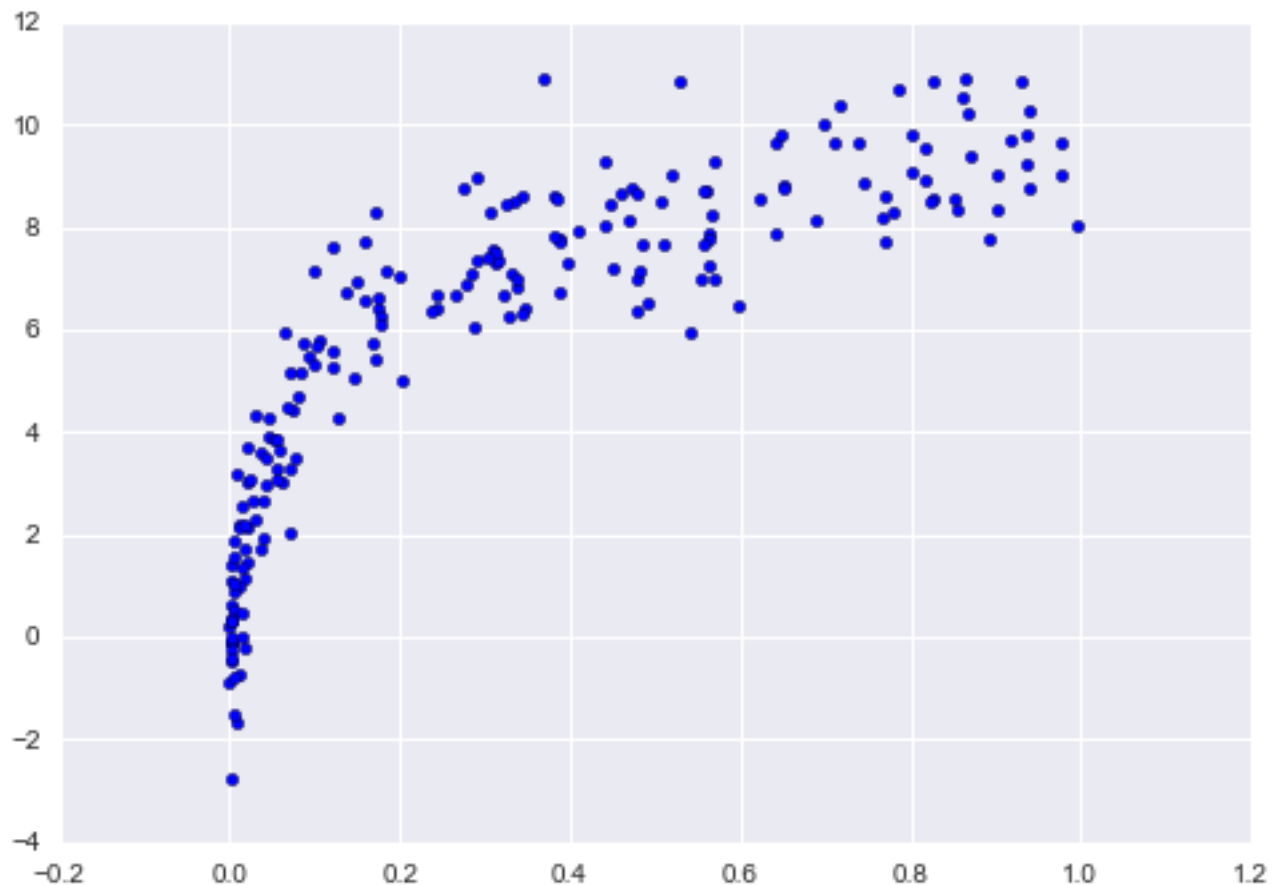


This shows precisely the qualitative behavior we expect: the training score is everywhere higher than the validation score; the training score is monotonically improving with increased model complexity; and the validation score reaches a maximum before dropping off as the model becomes over-fit.

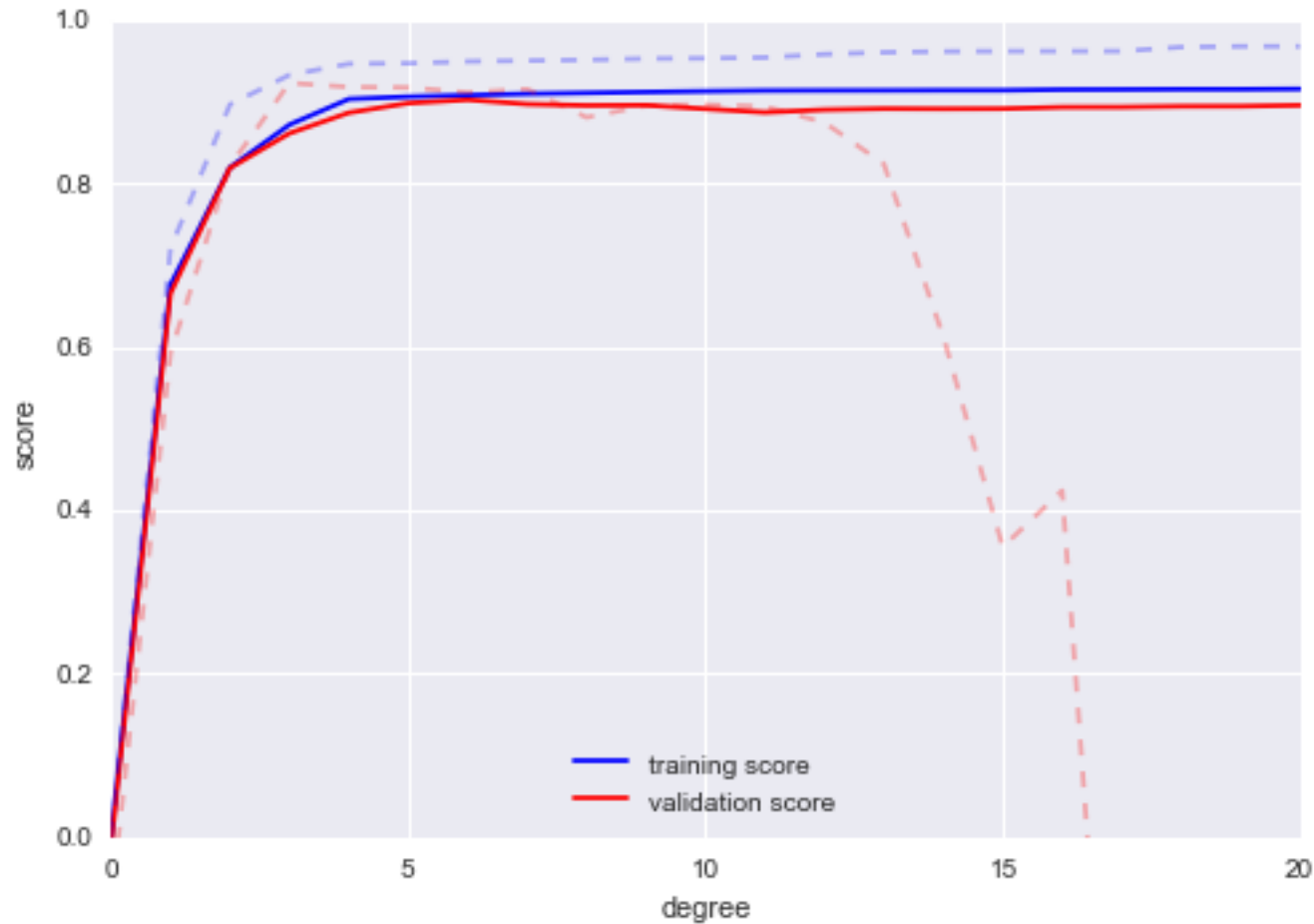
From the validation curve, we can read-off that the optimal trade-off between bias and variance is found for a third-order polynomial.

Learning Curves

One important aspect of model complexity is that the optimal model will generally depend on the size of your training data. For example, let's generate a new dataset with a factor of five more points:



Learning Curves



The solid lines show the new results, while the fainter dashed lines show the results of the previous smaller dataset. The peak here is probably around a degree of 6, but even a degree-20 model is not seriously over-fitting the data...

Learning Curves

A plot of the training/validation score with respect to the size of the training set is known as a learning curve.



Learning Curves

The general behavior we would expect from a learning curve is this:

- A model of a given complexity will overfit a small dataset: this means the training score will be relatively high, while the validation score will be relatively low.
- A model of a given complexity will underfit a large dataset: this means that the training score will decrease, but the validation score will increase.
- A model will never, except by chance, give a better score to the validation set than the training set: this means the curves should keep getting closer together but never cross.

The notable feature of the learning curve is the convergence to a particular score as the number of training samples grows. In particular, once you have enough points that a particular model has converged, adding more training data will not help you! The only way to increase model performance in this case is to use another (often more complex) model.

The Linear Regression

Supervised Learning

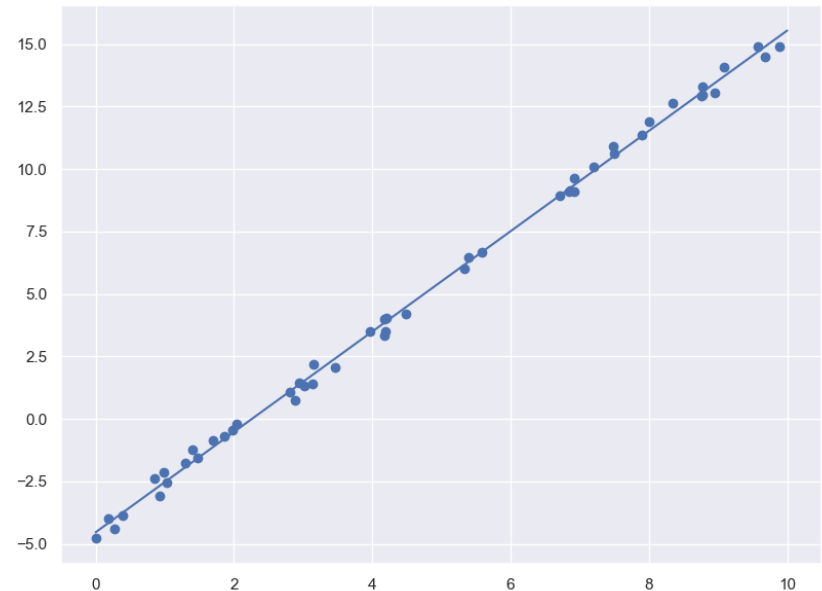
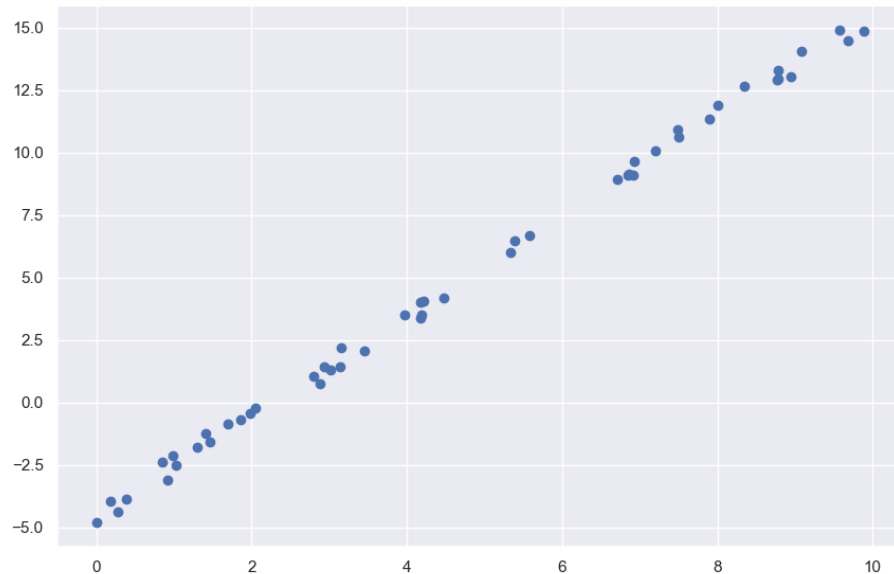
Simple Linear Regression

The most familiar linear regression consists of a straight line:

$$y = ax + b$$

where:

- a is the slope
- b is the intercept



Model slope: 2.02720881036
Model intercept: -4.99857708555

Supervised Learning

Linear Discriminants

The Linear Regression

Regression=predict an unknown value y from a set of inputs x_i .

We are thinking about multidimensional linear models:

$$y_j = \sum_{i=0}^M a_i x_i^{(j)}$$

where:

- a_0 is the intercept
- $a_{i>0}$ are the parameters of the line that best fit the data.
- $x_i^{(j)}$ is the i -th input of the j -th training example
- y_j is the prediction for the j -th training example

How defining these parameters a_i ?

Supervised Learning

Linear Discriminants

The Linear Regression

The most common solution is to minimise an error function that measures the sum of distances between the datapoints and the fitted line, that is:

$$\sum_{j=0}^N \left(t_j - \sum_{i=0}^M a_i x_i^{(j)} \right)^2$$

In matrix form: $(t - XA)^T(t - XA)$

Computing the smallest value of this means differentiating it with respect to the parameter vector and setting the derivative to 0, which means that $X^T(t - XA) = 0$

The solution is: $A = (X^T X)^{-1} X^T y$ (assuming that $X^T X$ is inverted).

For a given input vector z , the prediction is $A^T z$.

Supervised Learning

Linear Discriminants

Basis Function Regression

The idea is to take our multidimensional linear model:

$$y = a_0 + a_1x_1 + a_2x_2 + \dots$$

and build the features $x_1, x_2, \dots$ and so on, from our single-dimensional input x . We let $x_n = f_n(x)$, where $f_n()$ is some function that transforms our data.

For example, if $f_n(x) = x^n$, our model becomes a polynomial regression:

$$y = a_0 + a_1x + a_2x^2 + \dots$$

This is still a linear model: the linearity refers to the fact that the coefficients a_n never multiply or divide each other.

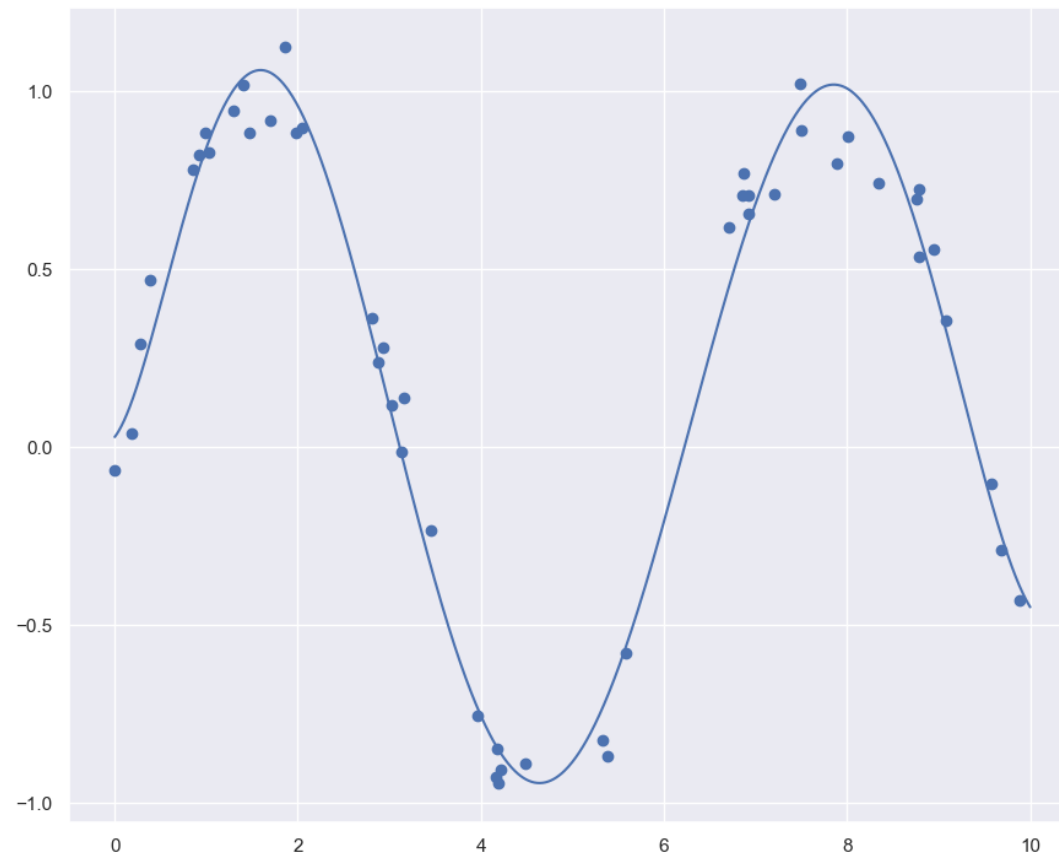
What we have effectively done is taken our one-dimensional x values and projected them into a higher dimension, so that a linear fit can fit more complicated relationships between x and y .

Supervised Learning

Linear Discriminants

Polynomial Basis Function

This linear model, through the use of 7th-order polynomial basis functions, can provide an excellent fit to this non-linear data...

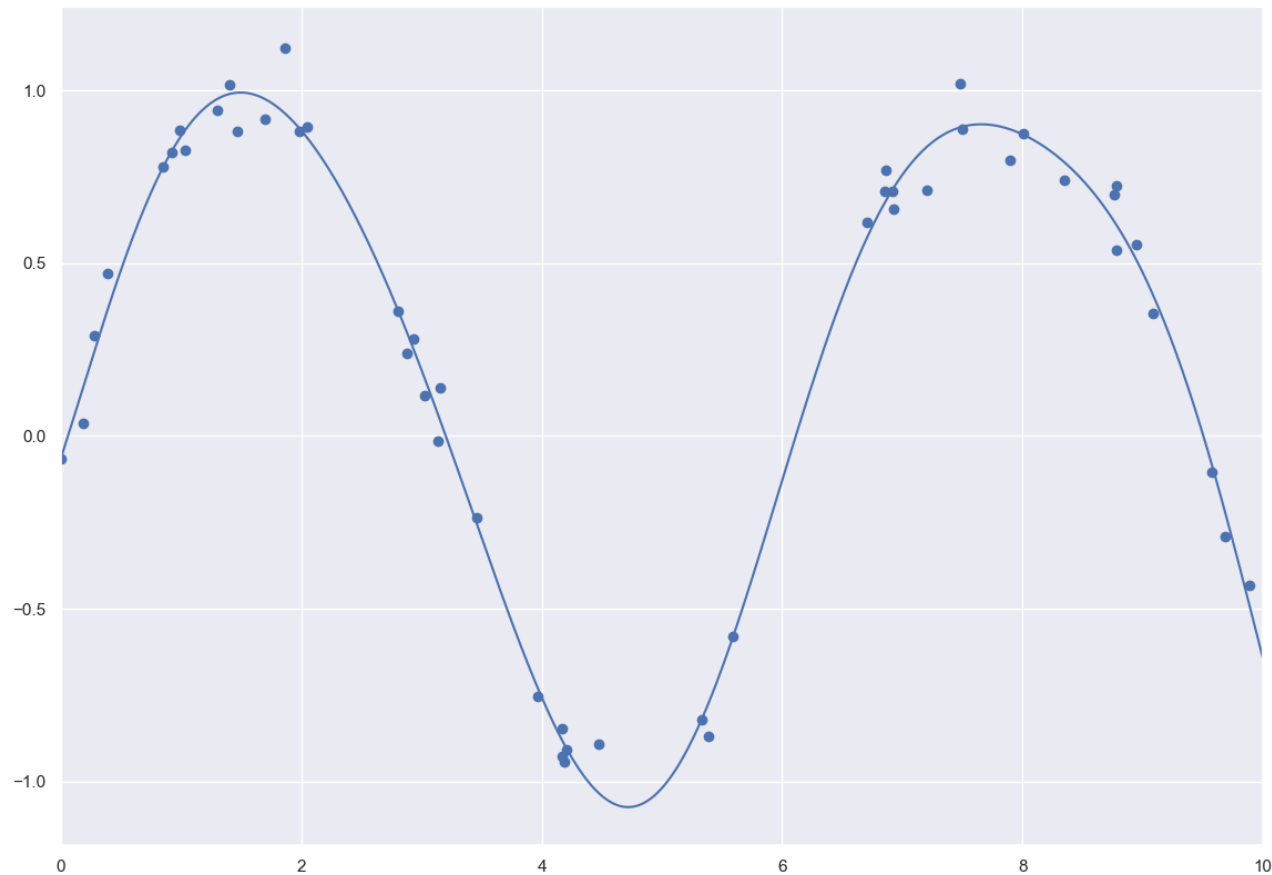


Supervised Learning

Linear Discriminants

Gaussian Basis Function

The smooth curve below is a sum of 10 Gaussian bases.

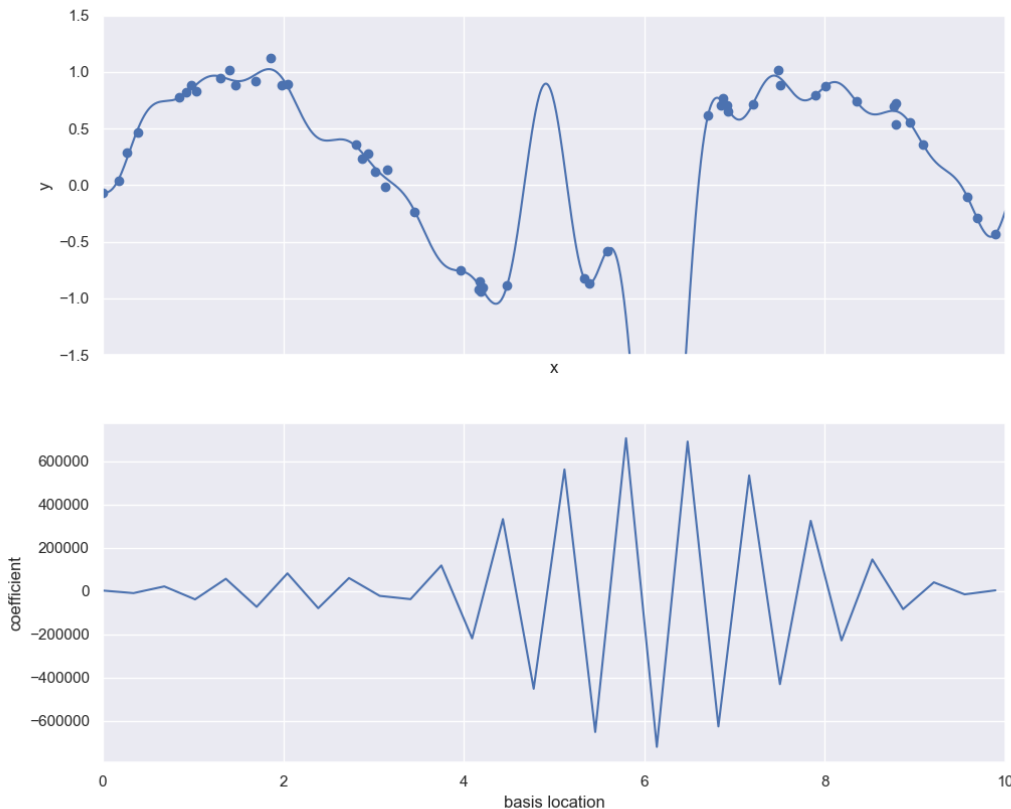


Supervised Learning

Linear Discriminants

Regularization

With the data projected to the 30-dimensional basis, the model has far too much flexibility and goes to extreme values between locations where it is constrained by data. We can see the reason for this if we plot the coefficients of the Gaussian bases with respect to their locations:



The lower panel of this figure shows the amplitude of the basis function at each location. This is typical over-fitting behavior when basis functions overlap: the coefficients of adjacent basis functions blow up and cancel each other out. We know that such behavior is problematic, and it would be nice if we could limit such spikes explicitly in the model by penalizing large values of the model parameters. Such a penalty is known as regularization, and comes in several forms.

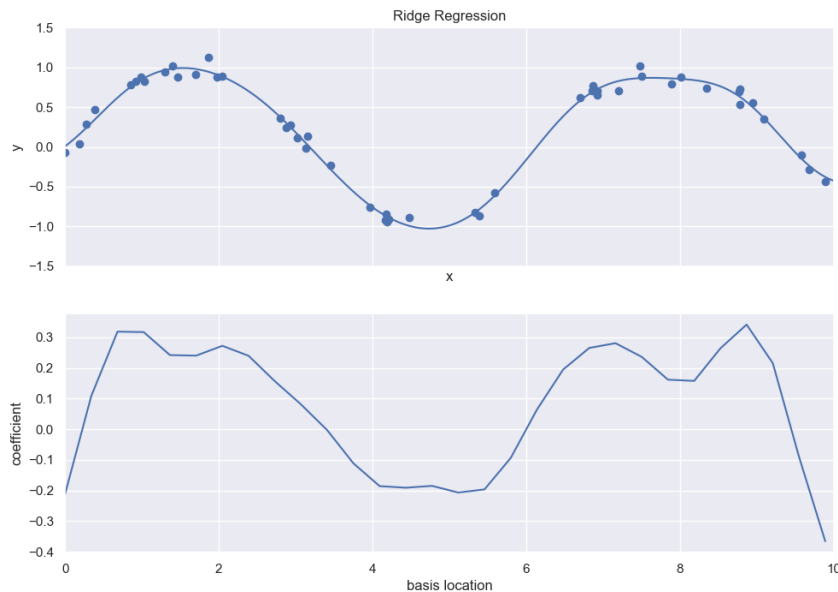
Supervised Learning

Ridge Regularization

Perhaps the most common form of regularization is known as ridge regression or L_2 regularization. This proceeds by penalizing the sum of squares (2-norms) of the model coefficients; in this case, the penalty on the model fit would be:

$$P = \alpha \sum_{n=1}^N \theta_n^2$$

where α is a free parameter that controls the strength of the penalty.



The α parameter is essentially a knob controlling the complexity of the resulting model. In the limit $\alpha \rightarrow 0$, we recover the standard linear regression result; in the limit $\alpha \rightarrow \infty$, all model responses will be suppressed.

Supervised Learning

Lasso Regularization

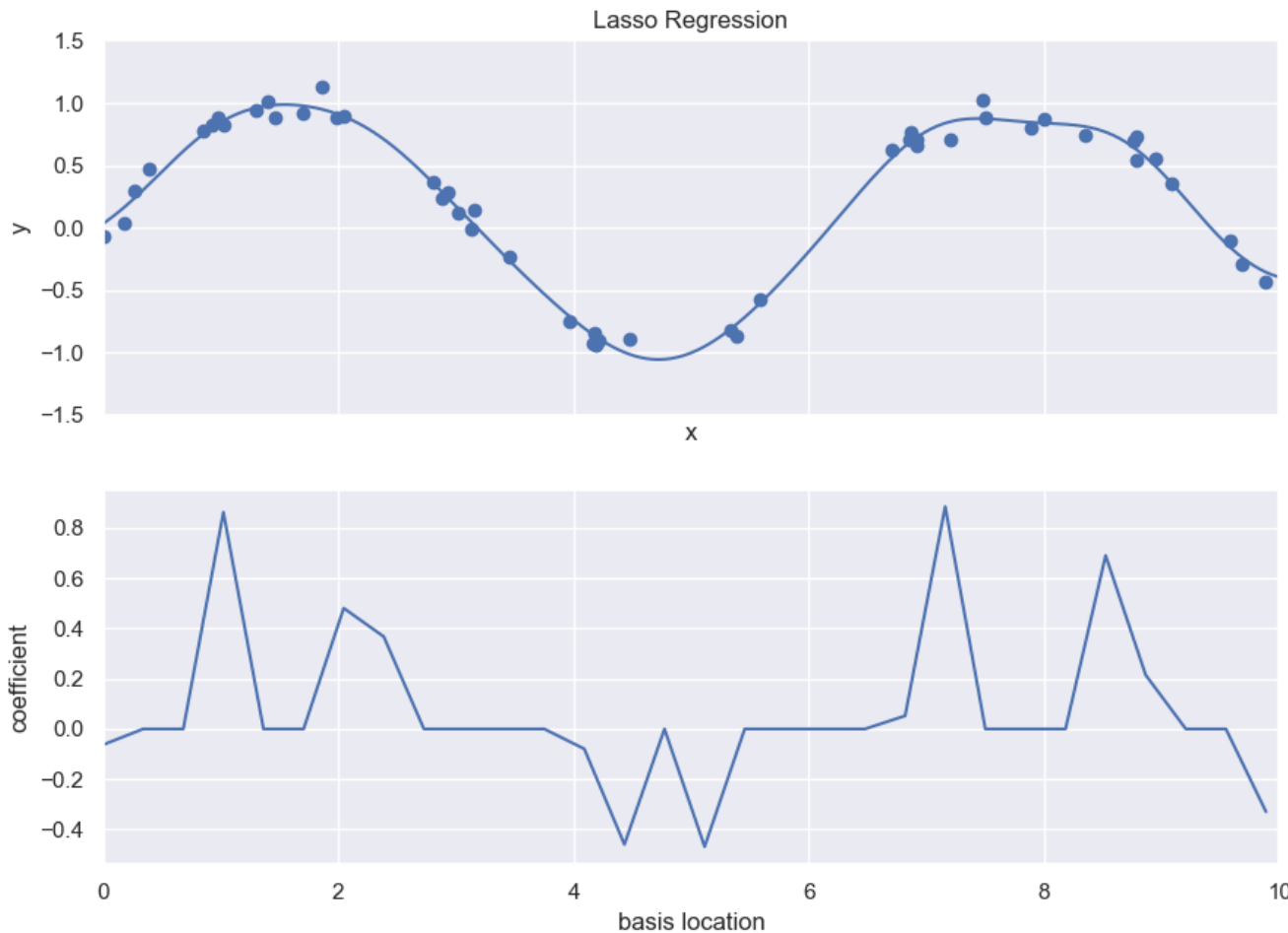
Another very common type of regularization is known as lasso, and involves penalizing the sum of absolute values (1-norms) of regression coefficients:

$$P = \alpha \sum_{n=1}^N |\theta_n|$$

Though this is conceptually very similar to ridge regression, the results can differ surprisingly. For example, due to geometric reasons lasso regression tends to favor sparse models where possible: that is, it preferentially sets model coefficients to exactly zero.

Supervised Learning

Lasso Regularization



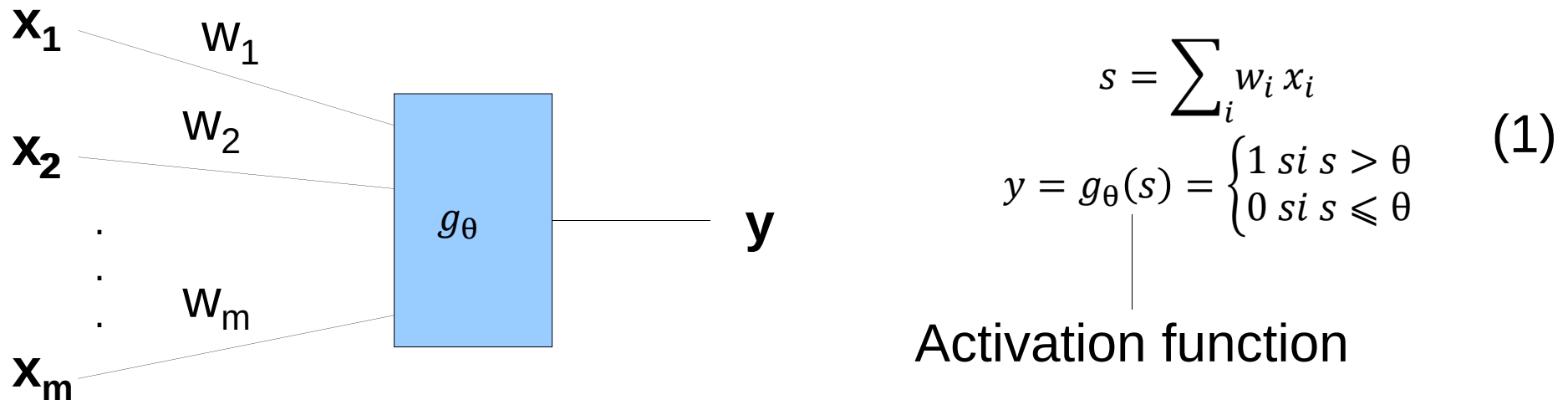
With the lasso regression penalty, the majority of the coefficients are exactly zero, with the functional behavior being modeled by a small subset of the available basis functions.

Mc Culloch and Pitts Neuronal Model

Supervised Learning

Classical neuronal model

The McCulloch and Pitt neuronal model



In order to make a neuron learn, how should we change the weights $\mathbf{w}$ and threshold θ of the neuron so that the network gets the right answer?

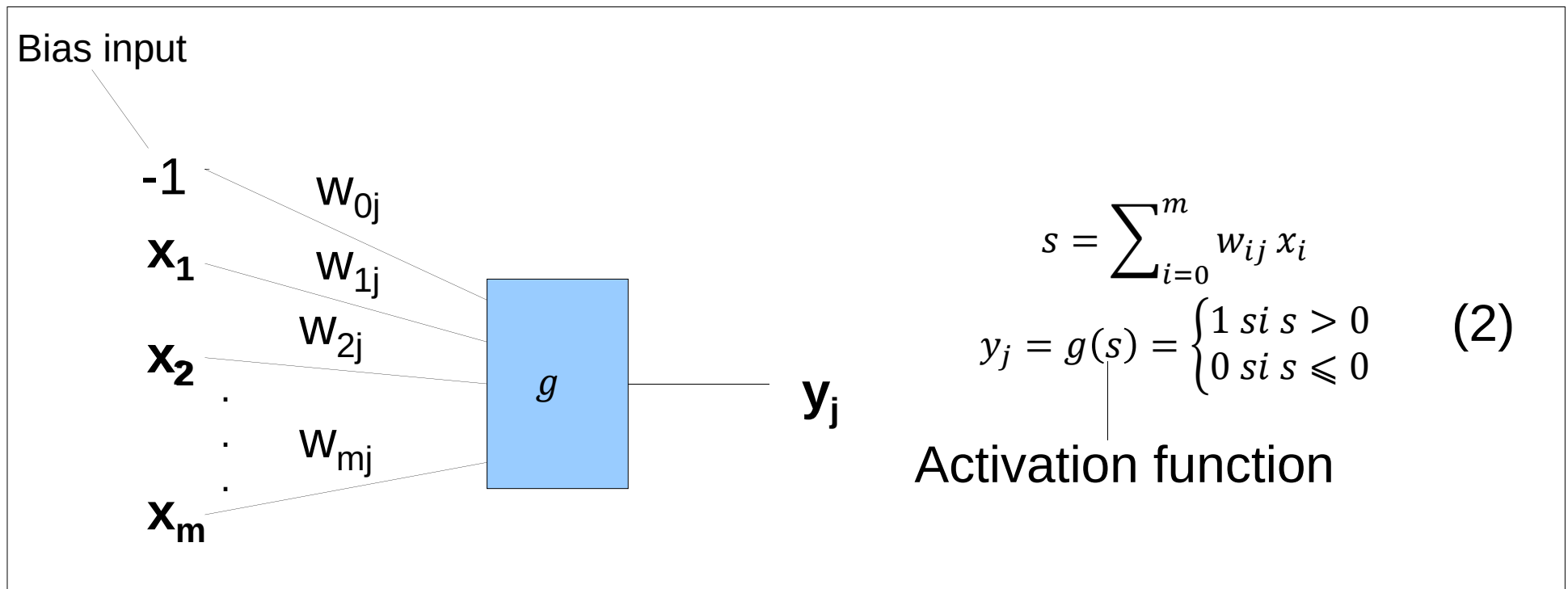
The Perceptron

Supervised Learning

Linear Discriminants

The Perceptron

The Perceptron is a collection of *McCulloch and Pitts* neurons with inputs, weights and outputs. w_{ij} is the weight of the connection between neuron/input i and neuron j . y_j is the output of neuron j .

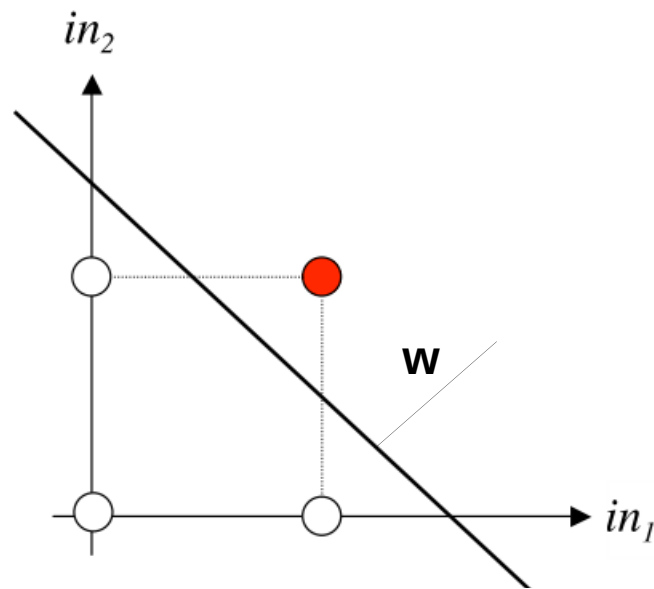


A Perceptron neuron

Supervised Learning

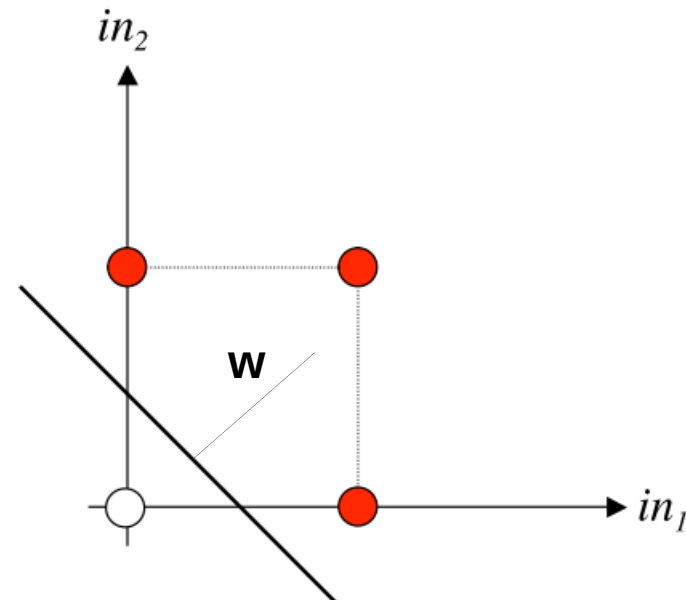
Decision Boundaries of AND and OR

Problems with input patterns that can be classified using a single hyperplane are said to be linearly separable.



AND

$$w_1 = 1, w_2 = 1, \theta = 1.5$$



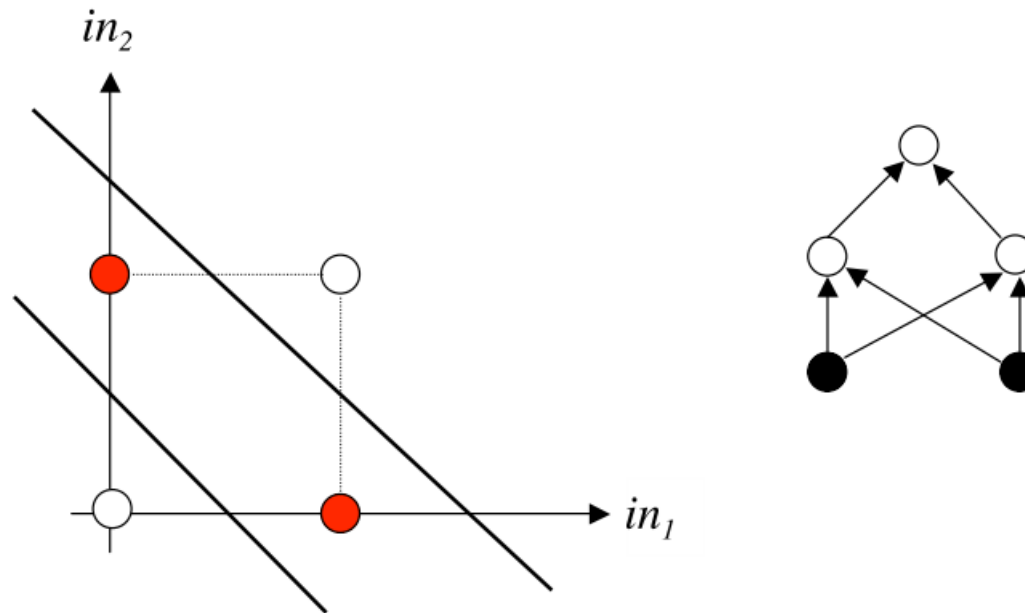
OR

$$w_1 = 1, w_2 = 1, \theta = 0.5$$

Supervised Learning

Decision Boundaries of XOR

Problems that cannot be classified this way are said to be non-linearly separable.



Supervised Learning

Decision hyperplanes and Linear Separability

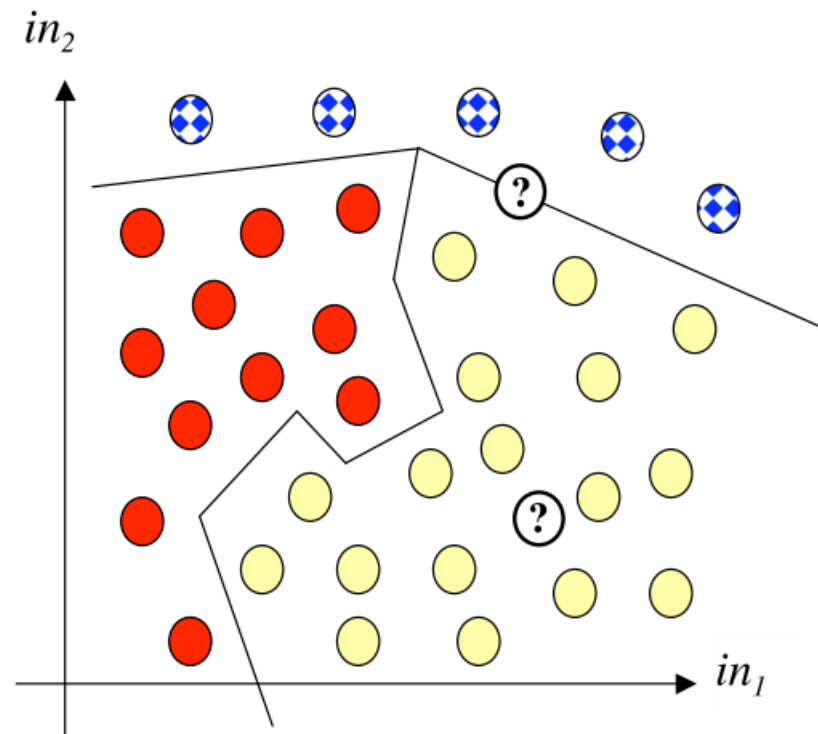
The weights define a decision boundary that is an $n-1$ dimensional *hyperplane* in the n dimensional input space:

$$\mathbf{w}^T \mathbf{x} - \theta = 0$$

This hyperplane is clearly still linear (i.e., straight or flat or non-curved) and can still only divide the space into two regions.

Supervised Learning

General Decision Boundaries



Supervised Learning

Learning and Generalization

A network will also produce outputs for input patterns that it was not originally set up to classify (shown with question marks), though those classifications may be incorrect.

There are two important aspects of the network's operation to consider:

Learning: The network must learn decision boundaries from a set of training patterns so that these training patterns are classified correctly.

Generalization: After training, the network must also be able to generalize, i.e. correctly classify test patterns it has never seen before.

Supervised Learning

Learning and Generalization

Usually we want the neural network to learn in a way that produces good generalization.

Sometimes, the training data may contain errors (e.g., noise in the input values, or incorrect classifications).

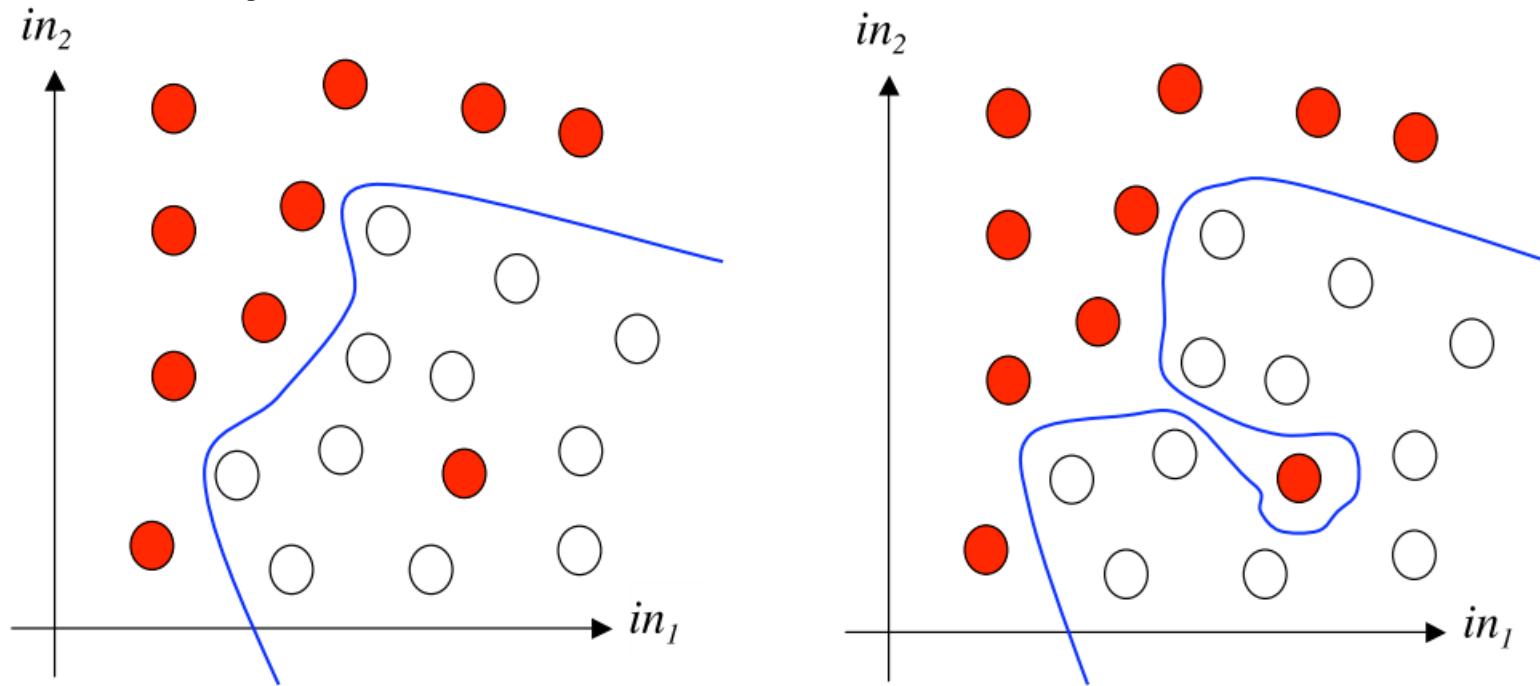
In this case, learning the training data perfectly may make the generalization worse.

There is an important trade-off between learning and generalization that arises quite generally.

Supervised Learning

Generalization in Classification

Suppose the task of the neural network is to learn a classification decision boundary:

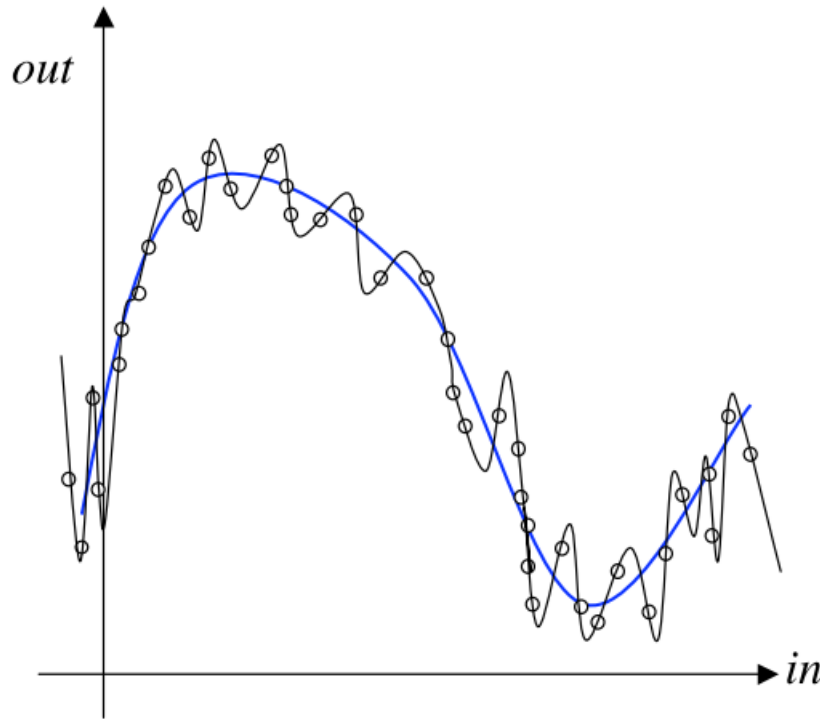


The aim is to get the network to generalize to classify new inputs appropriately. If we know that the training data contains noise, we don't necessarily want the training data to be classified totally accurately, as that is likely to reduce the generalization ability.

Supervised Learning

Generalization in Function Approximation

Suppose we wish to recover a function for which we only have noisy data samples:



We can expect the neural network output to give a better representation of the underlying function if its output curve does not pass through all the data points. Again, allowing a larger error on the training data is likely to lead to better generalization.

Supervised Learning

Training a Neural Network

We compute (3) for each neuron and finally get a vector of neurons that fired or not.

How to update the weights w_{ij} connected to a neuron j if its output is wrong?

Learning equation: $w_{ij} \leftarrow w_{ij} + \underbrace{\eta(t_j - y_j)}_{\text{error}} x_i \quad (3)$

Learning rate
 $\eta \in [0.1; 0.4]$

Input i

Supervised Learning

The Perceptron Algorithm

Initialisation

- Set all the weights to small (positive and negative) random numbers.

Training

- For T iterations:
 - For each input vector (training example):
 - Compute the activation of each neuron j using (2)
 - Update each of the weights individually using (3)

Recall

- Compute the activation of each neuron j using (2)

Supervised Learning Overview with Perceptron

1. Neural network classifiers learn decision boundaries from training data.
2. Simple Perceptrons can only cope with linearly separable problems.
3. Trained networks are expected to generalize, i.e. deal appropriately with input data they were not trained on.
4. One can train networks by iteratively updating their weights.
5. The Perceptron Learning Rule will find weights for linearly separable problems in a finite number of iterations.

Deep Neural Networks (DNN)

What problems a DNN can solve?

Four types of problems are generally solved using a DNN:

- Regression problems
- Classification problems
- Clustering problems
- Description problems
- Transduction problems
- Dimensionality reduction
- Time-series prediction

We have a set of data that shows how something varies over time.
We want to predict how the data will vary in the future.

$$y = x(t + \tau) = f(x(t), x(t - \tau), \dots, x(t - k\tau))$$

- Data Compression/Data denoising

Real-life applications of DNN

Applications where DNN can be used are numerous:

- Medical diagnosis
- Loan granting
- Self-driving cars (like ALVINN...)
- Handwriting characters recognition
- Stock market prediction
- Generating client profiles
- Describing the content of images or videos
(example of a description problem)
- Translating sentences of a language into sentences of another one
(example of a transduction problem)
- Converting 4K images/videos into 256-colored images/videos
- ...

Supervised Learning

DNN=Multi-layer Neural Networks

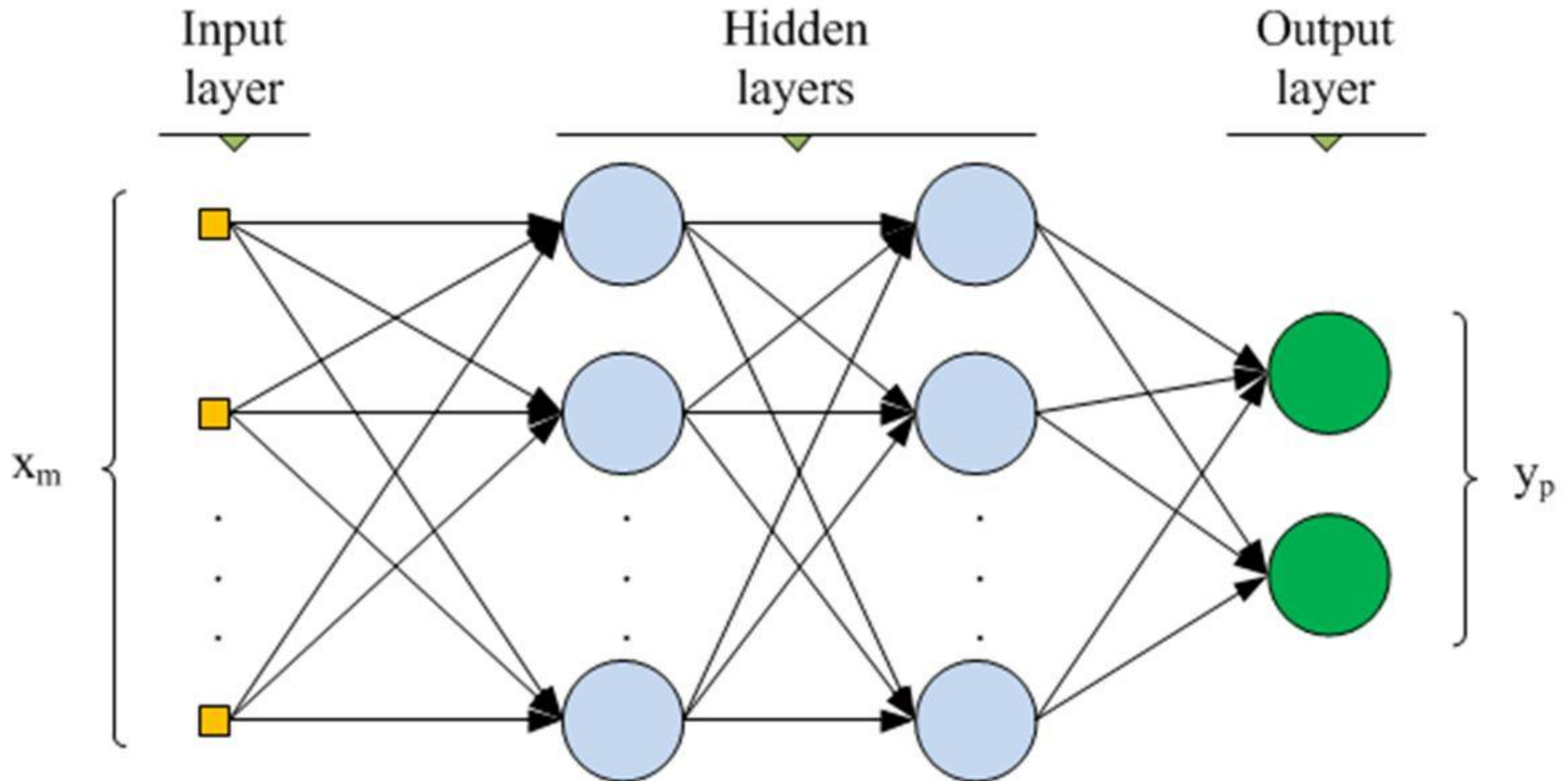
The majority of interesting problems are not linearly separable.
Some problems can be made linearly separable if we can work out how to transform the features.

Learning in neural networks consists in changing the weights.
More complex neural networks mean performing more computation which implies adding more weights.

At least three solutions:

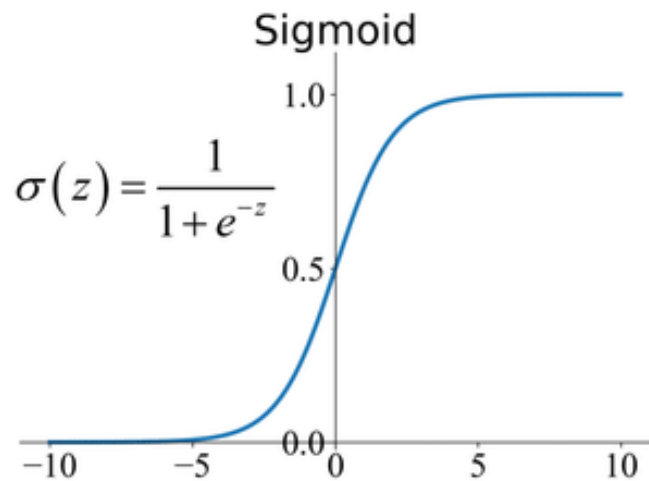
- add new neurons in several layers between the inputs and outputs.
→ Deep Neural Networks
- add some backward connections, so that the output neurons connect to the inputs again.
→ Recurrent Neural Networks
- add parameters to better recognize patterns in the data.
→ Convolutional Neural Networks

Supervised Learning Deep Neural Networks (DNN)

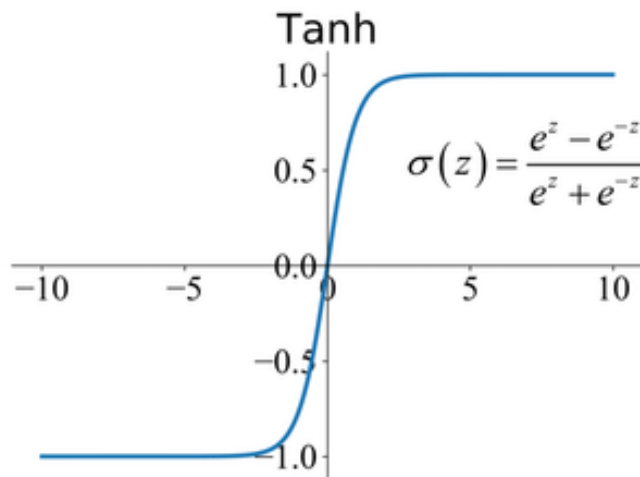


Supervised Learning

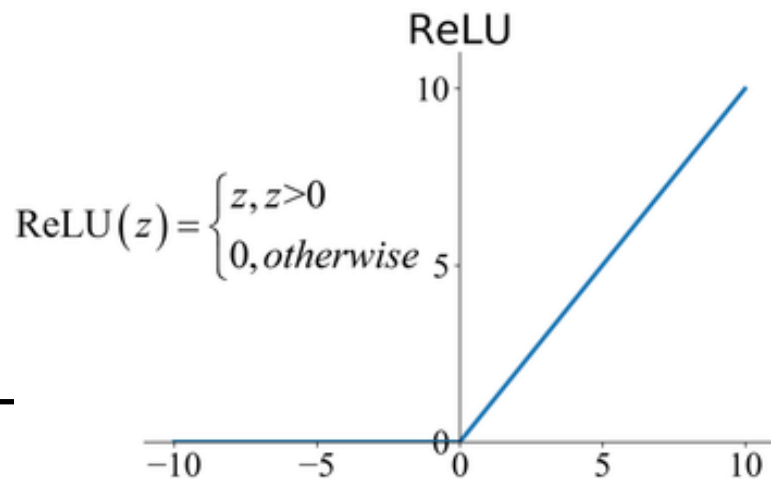
Activation functions (for neurons except output neurons)



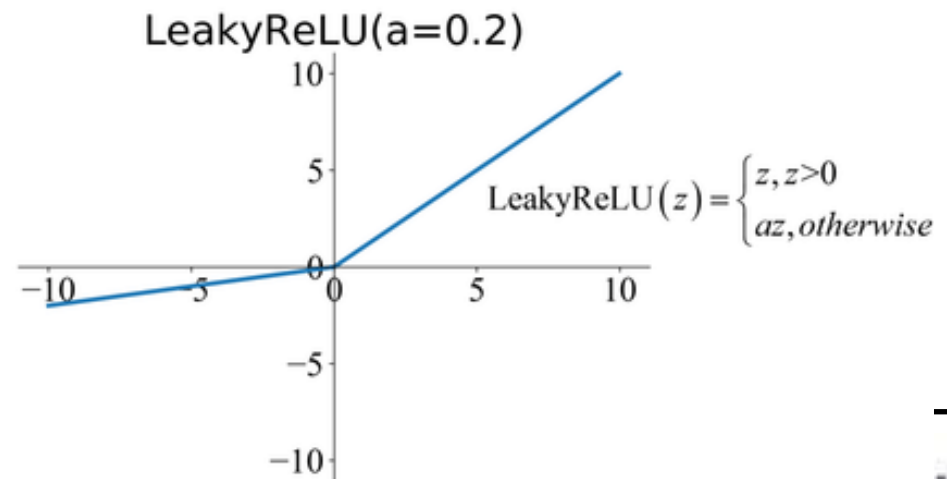
(a)



(b)



(c)



(d)

Supervised Learning

A word about several specificities

For a classification problem:

- Binary:
 - Loss=*binary_crossentropy*

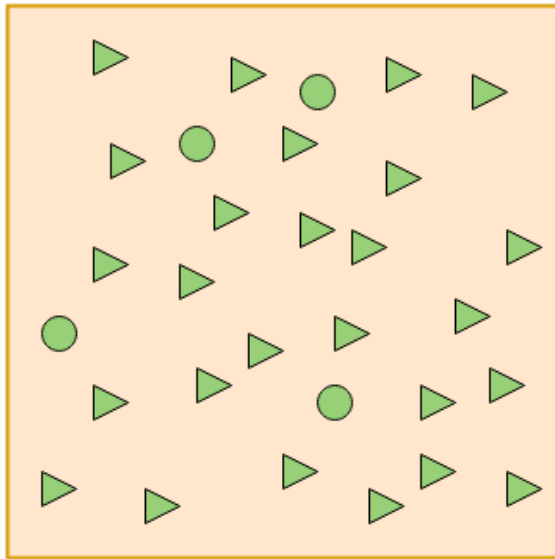
$$H(X) = \begin{cases} - \int_x p(x) \log p(x), & \text{if } X \text{ is continuous} \\ - \sum_x p(x) \log p(x), & \text{if } X \text{ is discrete} \end{cases}$$

- Activation function of the output of the last layer: *sigmoid*

Binary Cross-Entropy

Consider the following 3 “containers” with shapes: triangles and circles

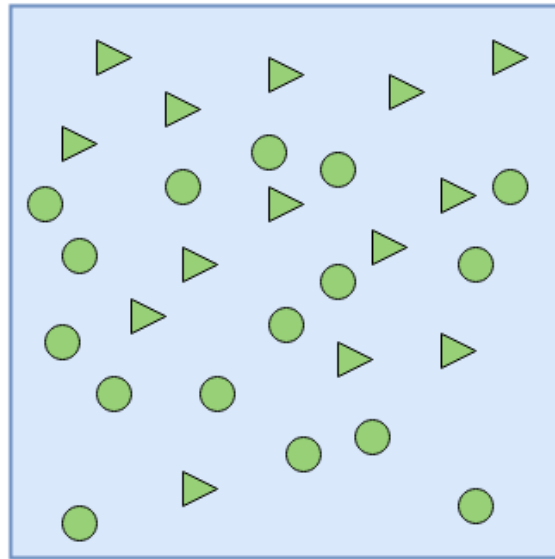
Container 1



#▶ = 26, #● = 4

Container 1: The probability of picking a triangle is $26/30$ and the probability of picking a circle is $4/30$. For this reason, the probability of picking one shape and/or not picking another is more certain.

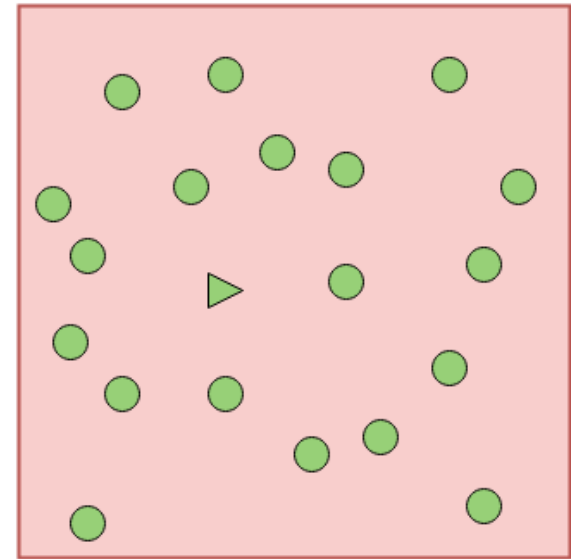
Container 2



#▶ = 14, #● = 16

Container 2: Probability of picking the a triangular shape is $14/30$ and $16/30$ otherwise. There is almost 50–50 chance of picking any particular shape. Less certainty of picking a given shape than in 1.

Container 3



#▶ = 1 #● = 29

Container 3: A shape picked from container 3 is highly likely to be a circle. Probability of picking a circle is $29/30$ and the probability of picking a triangle is $1/30$. It is highly certain than the shape picked will be circle.

Entropy for container 1:

$$\begin{aligned} H(X) &= - \sum_x p(x) \log(p(x)) \\ &= -[p(x_1) \log_2(p(x_1)) + p(x_2) \log_2(p(x_2))] \\ &= - \left[\frac{26}{30} \log_2 \left(\frac{26}{30} \right) + \frac{4}{30} \log_2 \left(\frac{4}{30} \right) \right] \\ &= 0.5665 \end{aligned}$$

Entropy for container 2:

$$\begin{aligned} H(X) &= - \sum_x p(x) \log(p(x)) \\ &= -[p(x_1) \log_2(p(x_1)) + p(x_2) \log_2(p(x_2))] \\ &= - \left[\frac{14}{30} \log_2 \left(\frac{14}{30} \right) + \frac{16}{30} \log_2 \left(\frac{16}{30} \right) \right] \\ &= 0.9968 \end{aligned}$$

Entropy for container 3:

$$\begin{aligned} H(X) &= - \sum_x p(x) \log(p(x)) \\ &= -[p(x_1) \log_2(p(x_1)) + p(x_2) \log_2(p(x_2))] \\ &= - \left[\frac{1}{30} \log_2 \left(\frac{1}{30} \right) + \frac{29}{30} \log_2 \left(\frac{29}{30} \right) \right] \\ &= 0.2108 \end{aligned}$$

Supervised Learning

A word about several specificities

For a classification problem:

- Multiple:
 - Loss=*categorical_crossentropy*

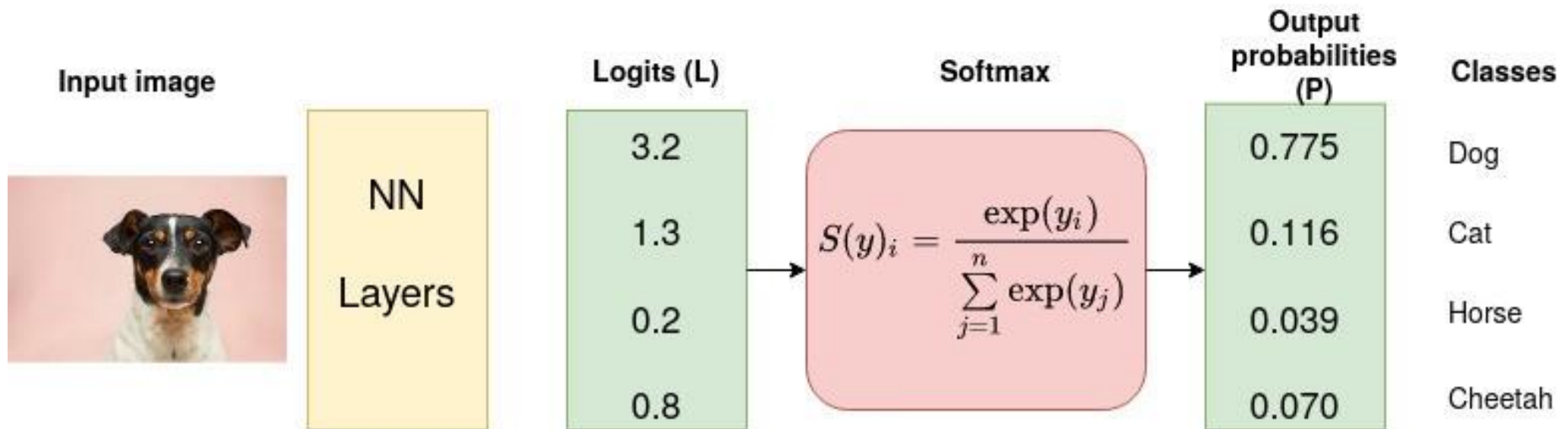
$$L_{\text{CE}} = - \sum_{i=1}^n t_i \log(p_i), \text{ for } n \text{ classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the i^{th} class.

- Activation function of the output of the last layer: *softmax*

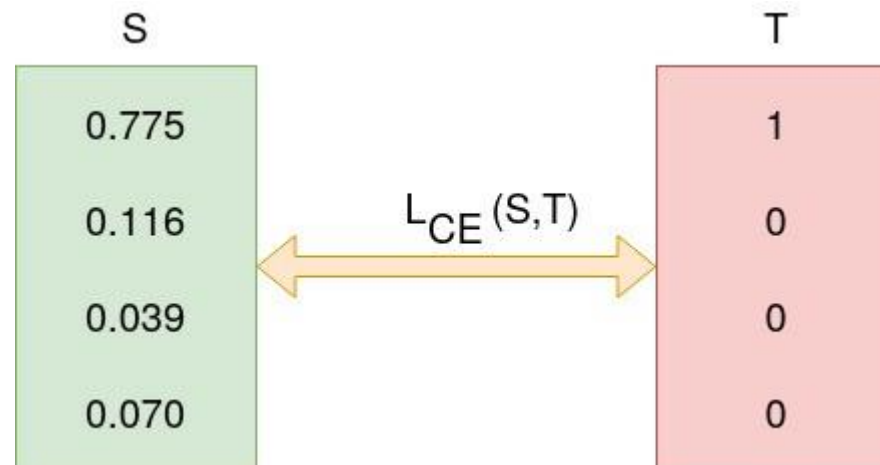
$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

An example of using *Softmax* for image classification



Categorical Cross-Entropy

Consider the classification problem with the following Softmax probabilities (S) and the labels (T).



$$\begin{aligned} L_{CE} &= - \sum_{i=1} T_i \log(S_i) \\ &= - [1 \log_2(0.775) + 0 \log_2(0.126) + 0 \log_2(0.039) + 0 \log_2(0.070)] \\ &= - \log_2(0.775) \\ &= 0.3677 \end{aligned}$$

Supervised Learning

A word about several specificities

- For a regression problem:
 - Loss=*mean_squared_error*

$$E(t, y) = \frac{1}{2} \sum_{j=1}^N (t_j - y_j)^2$$

- Activation function of the output of the last layer: *identity*
 $g(z)=z$

Supervised Learning

Training=Going forwards and backwards

Training a DNN means:

- computing the outputs for the given inputs and the current weights
→ Going forwards through the network
- updating the weights according to the **error** (difference between the target **t** and the prediction **y**)
→ Going backwards through the network

Supervised Learning

Going backwards

The **back-propagation of error** consists in sending the errors backwards through the network, by updating the weights so that the network makes better predictions in the future.

Three elements to understand back-prop:

- the derivative of $\frac{1}{2} x^2$
- 0 is the derivative of a function of x with respect to t
- the chain rule (how to differentiate composite functions)

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

With extra layers of neurons, minimising the error so that neurons only fire when they should (and do not fire when they shouldn't) is a harder problem than with no layer.

Chain rule

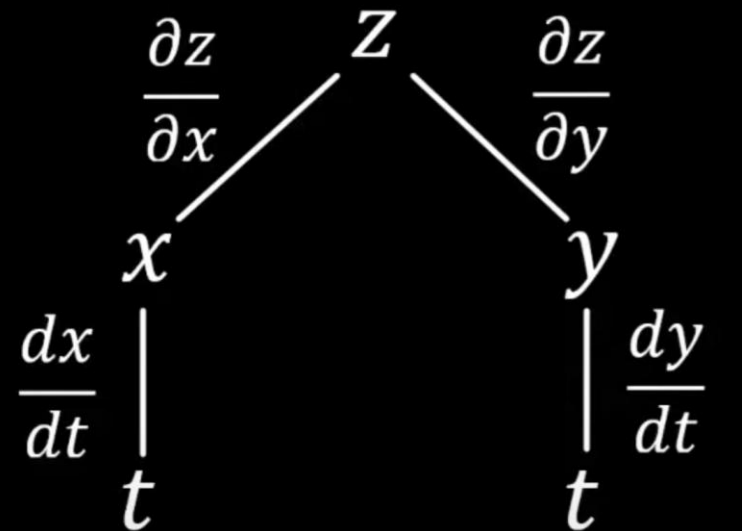
Given $z = f(x, y)$ the total differential for z is

$$dz = f_x dx + f_y dy$$

$$x = g(t) \quad y = h(t)$$

$$\frac{dz}{dt} = f_x \frac{dx}{dt} + f_y \frac{dy}{dt}$$

$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \frac{dx}{dt} + \frac{\partial z}{\partial y} \frac{dy}{dt}$$



$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \frac{dx}{dt} + \frac{\partial z}{\partial y} \frac{dy}{dt}$$

Chain rule: example 1

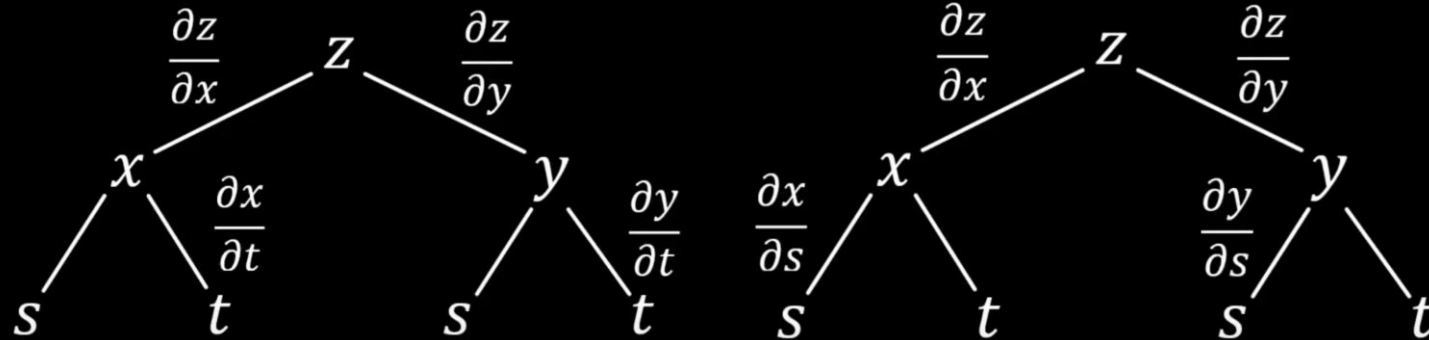
Given $z = x^3 + y^3$, $x = 2 \sin t$, $y = 3 \cos t$,
Calculate $\frac{dz}{dt}$

$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \cdot \frac{dx}{dt} + \frac{\partial z}{\partial y} \cdot \frac{dy}{dt}$$

$$\begin{aligned} &= (3x^2 + 0)(2 \cos t) + (0 + 3y^2)(-3 \sin t) \\ &= 6x^2 \cos t - 9y^2 \sin t \\ &= 6(2 \sin t)^2 \cos t - 9(3 \cos t)^2 \sin t \\ &= 24 \sin^2 t \cos t - 81 \cos^2 t \sin t \end{aligned}$$

Chain rule: example 2

$$z = f(x, y) \quad x = g(s, t) \quad y = h(s, t)$$



$$\frac{\partial z}{\partial t} = \frac{\partial z}{\partial x} \frac{\partial x}{\partial t} + \frac{\partial z}{\partial y} \frac{\partial y}{\partial t}$$

$$\frac{\partial z}{\partial s} = \frac{\partial z}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial z}{\partial y} \frac{\partial y}{\partial s}$$

Given $z = x^2 y$, $x = s + 2t$, $y = 2s - t$,

Calculate $\frac{\partial z}{\partial s}$ and $\frac{\partial z}{\partial t}$

Chain rule: example 2

$$\begin{aligned}\frac{\partial z}{\partial s} &= \frac{\partial z}{\partial x} \cdot \frac{\partial x}{\partial s} + \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial s} = y(2x)(1+0) + x^2(1)(2+0) \\ &= 2xy + 2x^2 \\ &= 2(s+2t)(2s-t) + 2(s+2t)^2 \\ &= 2(2s^2 - \underline{st} + \underline{4st} - 2t^2) + 2(s^2 + \underline{2st} + \underline{2st} + 4t^2) \\ &= \underline{4s^2} + \underline{6st} - \underline{4t^2} + \underline{2s^2} + \underline{8st} + \underline{8t^2} \\ &= 6s^2 + 14st + 4t^2 \\ &= 2(3s^2 + 7st + 2t^2) \\ &= 2(3s+t)(s+2t)\end{aligned}$$

Chain rule: example 2

$$\frac{\partial z}{\partial t} = \frac{\partial z}{\partial x} \cdot \frac{\partial x}{\partial t} + \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial t} = (2xy)(2) + (x^2)(-1) \\ = 4xy - x^2$$

$$= 4(s+2t)(2s-t) - (s+2t)^2$$

$$= 4(2s^2 - \underline{st} + \underline{4st} - 2t^2) - (s^2 + \underline{2st} + \underline{2st} + 4t^2)$$

$$= 8s^2 + 12st - 8t^2 - s^2 - 4st - 4t^2$$

$$= 7s^2 + 8st - 12t^2$$

$$= (7s - 6t)(s + 2t)$$

Supervised Learning

Error function

The sum-of-squares error function is the most common used:

$$E(t, y) = \frac{1}{2} \sum_{j=1}^N (t_j - y_j)^2$$

The derivative of E with respect to the weights is the gradient of the error.

To update each weight in the network :

- go forward through the network from a given input vector
- compute the output
- compute the error
- compute the gradient of this error and use it

Computing the gradient consists in derivating an error function, which can be done if the activation function is continuous...

Supervised Learning

DNN Algorithm

Initialisation

- Initialise all weights to small (positive and negative) random values

Training

- Repeat

- For each input vector

Forwards step

- Compute the activation of each neuron j in the hidden layer

$$h_j = \sum_i x_i v_{ij} \quad a_j = g(h_j) = \frac{1}{1 + \exp(-\beta h_j)}$$

- Work through the network until you get to the output layers, which have activations:

$$h_k = \sum_j a_j w_{jk} \quad y_k = g(h_k) = \frac{1}{1 + \exp(-\beta h_k)}$$

Supervised Learning

DNN Algorithm

Backwards step

- Compute the error at the output:

$$\delta_{ok} = (t_k - y_k)y_k(1 - y_k)$$

- Compute the error in the hidden layer(s):

$$\delta_{hj} = a_j(1 - a_j) \sum_k w_{jk} \delta_{ok}$$

- Update the output layer weights:

$$w_{jk} \leftarrow w_{jk} + \eta \delta_{ok} a_j$$

- Update the hidden layer weights:

$$v_{ij} \leftarrow v_{ij} + \eta \delta_{hj} x_i$$

- Randomize the order of the input vectors so that training is not always done in the same order each iteration

- Until Learning stops

Supervised Learning

DNN Algorithm details

- Initializing the weights

Uniform learning consists in setting the weights w randomly such that :

$$w \in \left[\frac{-1}{\sqrt{(n)}}; \frac{1}{\sqrt{(n)}} \right] \text{ where } n \text{ is the number of nodes in the input layer}$$

- Different output activation functions

- For classification: sigmoid/softmax function
- For regression: identity function $g(h)=h$

- *Stochastic/Batch* Gradient Descent (GD)

- Full GD: Weights are updated after each epoch (pass through all the training examples)
- Stochastic: Weights are updated after each new training example
- Batch: Weights are updated after each N new training examples

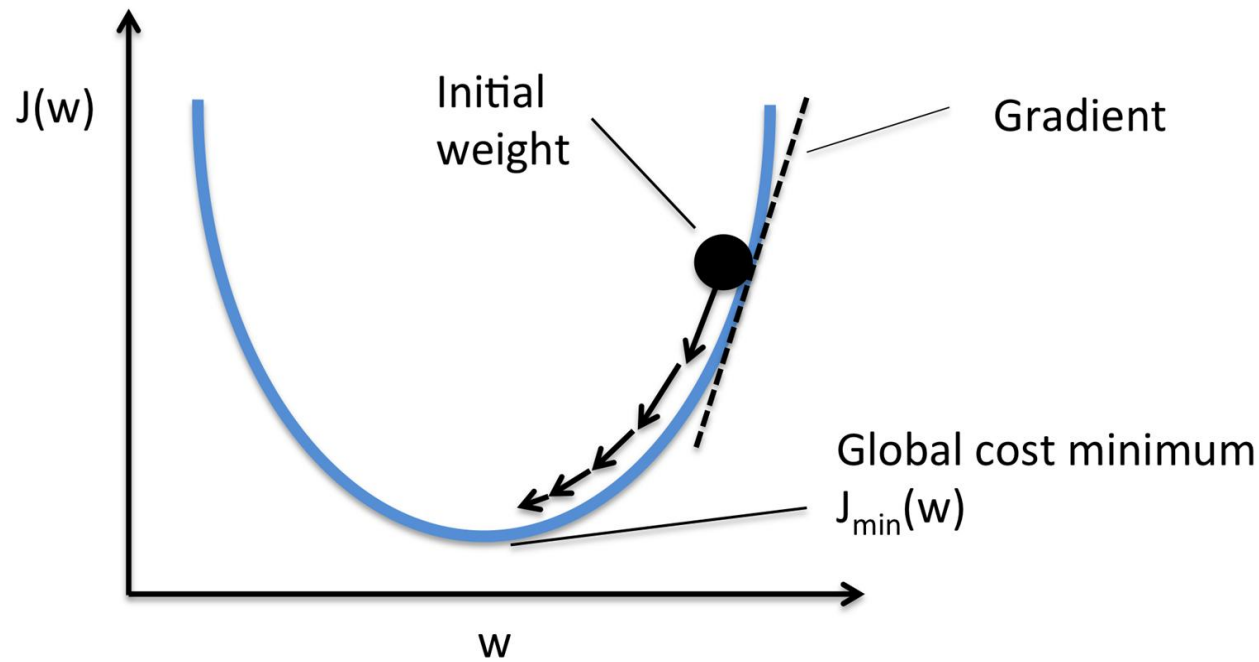
Supervised Learning

DNN Algorithm details

- Local Minima

In the network, error is minimized with gradient descent: we approximate the gradient of the error and follow it downhill so that we reach the bottom of the slope.

Following downhill only guarantees we can reach a local minimum.



Supervised Learning

DNN Algorithm details

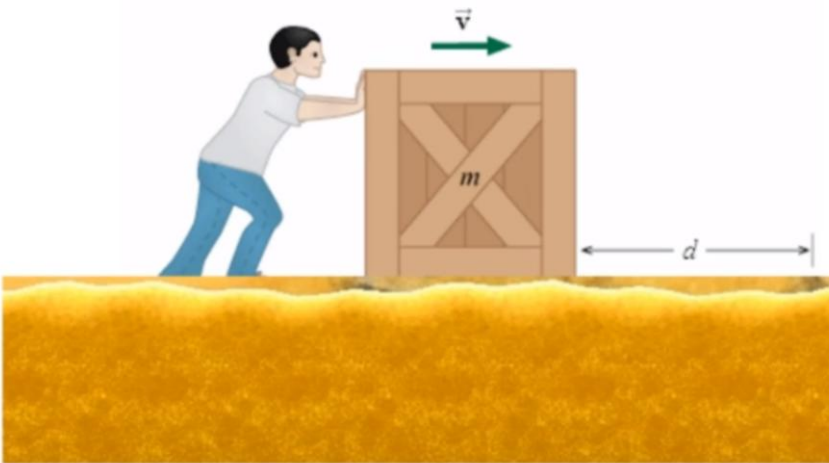
- Picking up momentum

Learning can be more stable by reducing the learning rate and adding a momentum:

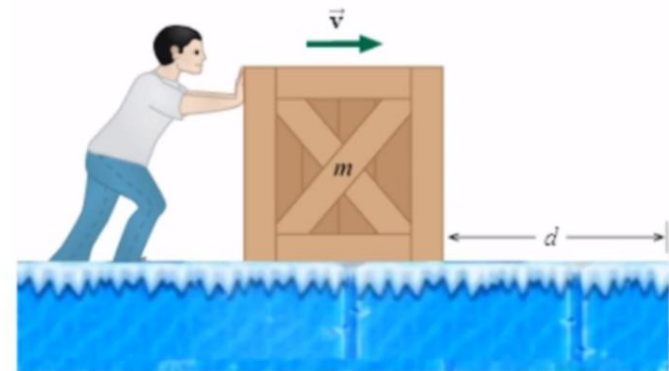
$$w_{ij}^t = w_{ij}^{t-1} + \eta \delta_o a_j + \alpha \Delta w_{ij}^{t-1}$$

$$0 < \alpha < 1$$

Push the box once, it continues to move, without additional force!



Without momentum



With momentum

Supervised Learning

DNN in Practice

- Data Preparation

Scaling the inputs and outputs or encoding in 1-To-N for outputs
In Python:

`data=(data-data.mean(axis=0))/data.var(axis=0)`

Same for outputs, or: 1-to-N encodings of targets

- Amount of Training Data

There are $(m+1)*n+(n+1)*p$ weights for DNN with one hidden layer, where: m, n, p are the number of nodes in the input, hidden, output layer.

Rule of thumb: # training examples=10 times the number of weights

- Number of hidden layers

Any smooth functional mapping can be approximated by a two-hidden-layer Perceptron with linear combination of localised sigmoidal functions.

Bad news: no theory to help choosing the right number of weights per layer.

Choose the one that gives the best results.

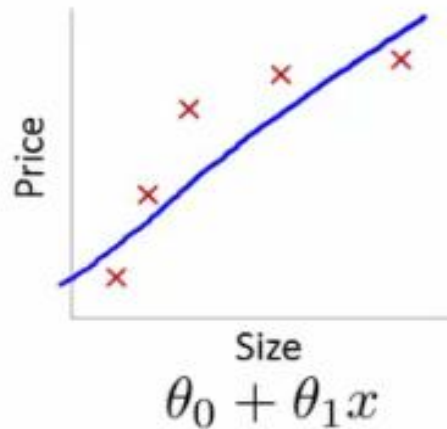


Supervised Learning

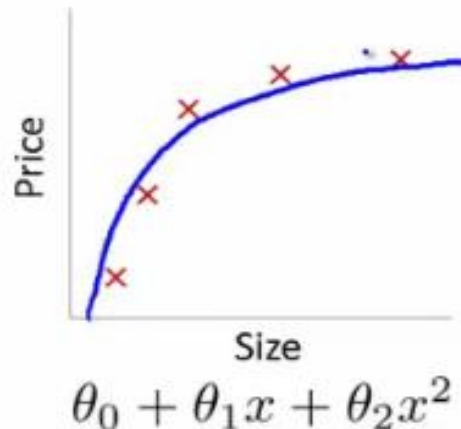
DNN in Practice

- Generalisation and Overfitting

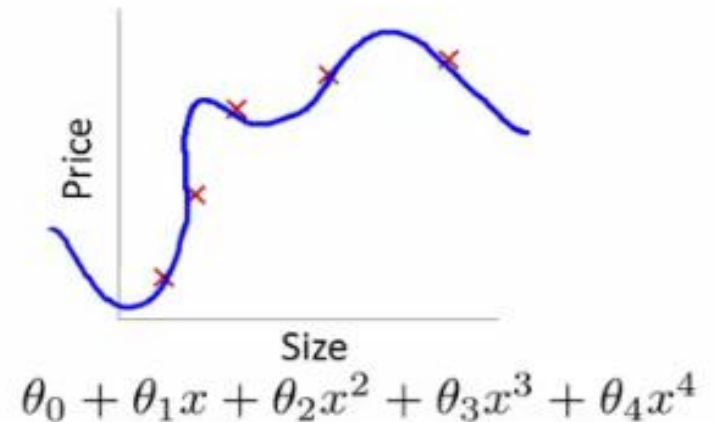
The whole purpose of a NN is to generalise from training examples to all possible inputs.



High bias
(underfit)



“Just right”

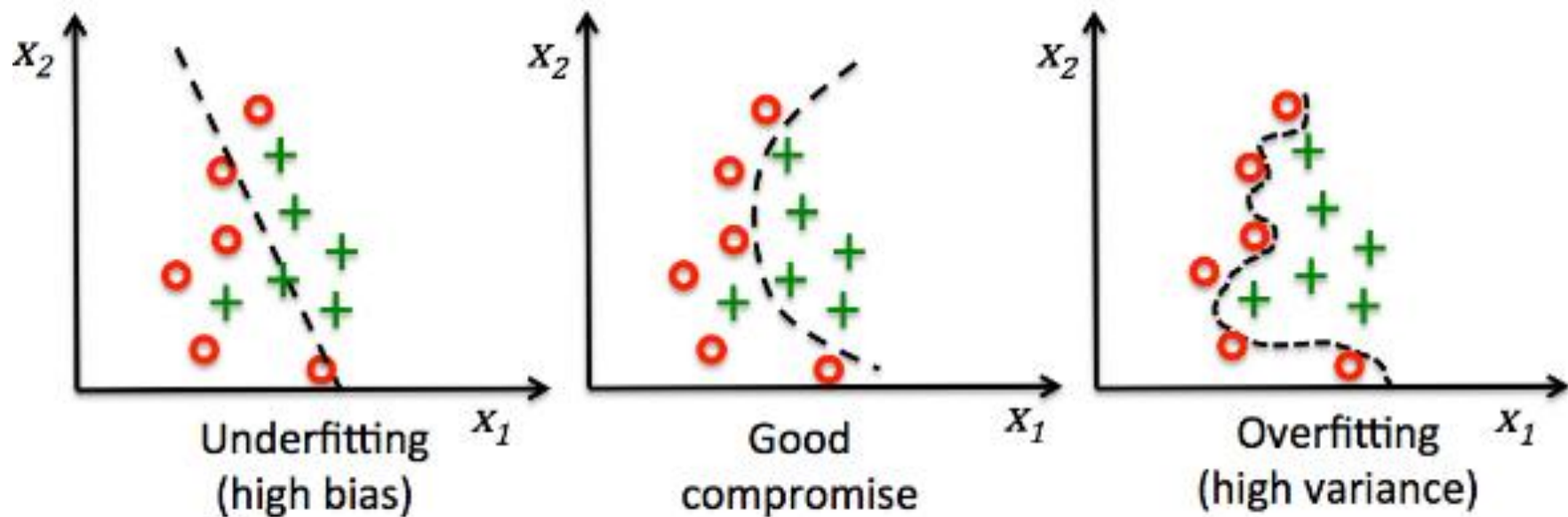


High variance
(overfit)

Supervised Learning

DNN in Practice

- Generalisation and Overfitting



Supervised Learning

DNN in Practice

- Training, testing and validation

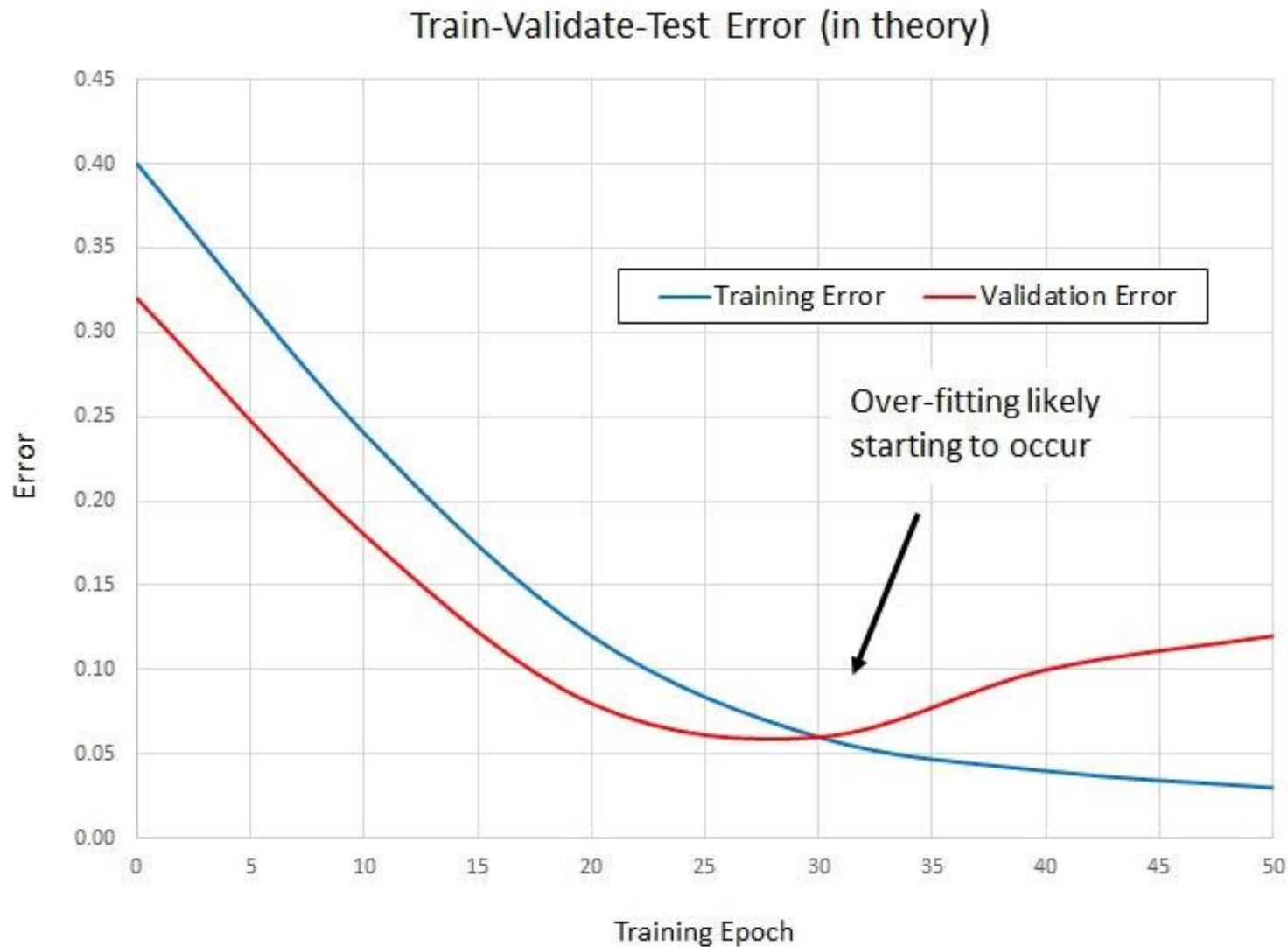
Three sets of data are usually used for learning an MLP:

- Training set is used to train it
- Test set is used to evaluate its generalisation capabilities
- A validation set may be used during training to estimate how well a DNN is generalising and deciding to stop learning when it starts increasing (early stopping).

Supervised Learning

DNN in Practice

- Training, testing and validation



Supervised Learning

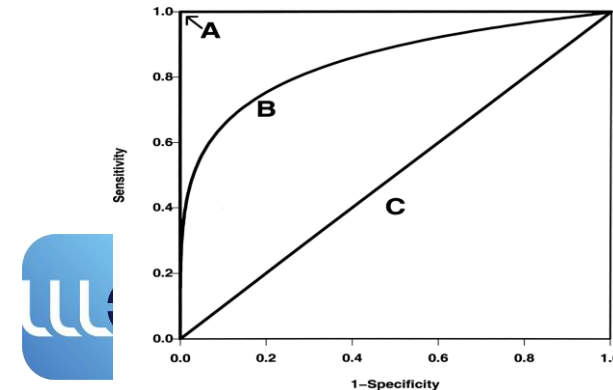
DNN in Practice

- Computing and evaluating the results

Reporting and analysing the data involves running the network forward on the test data.

Three ways to compute the results:

- For a classification problem:
 - Confusion matrix: compute the number of cases that the network predicted correctly for each class.
 - Precision: percentage of examples correctly classified
 - ROC curve: plot of the percentage of true positives (things correctly put into class 1) on the y axis against false positives (things incorrectly put into class 1) on the x axis.
- For a regression problem:
 - sum-of-squares error



Supervised Learning Overview with DNN

- Select inputs and outputs for your problem
 - Do you have enough data for the problem?
 - What features to use?
 - Output activation function?
- Normalise inputs (and outputs)
- Split the data into training, testing and validation sets
 - The ratio is often 50:25:25 or 60:20:20
 - When not enough data: (leave-one-out) cross-validation
- Select a network architecture
- Train the network
- Test it